Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Kierunek: Informatyka

Igor Barcikowski

Aplikacja do zarządzania rezerwacjami

Praca inżynierska wykonana
w katedrze

pod kierunkiem
promotora

Olsztyn, 2026 rok

University of Warmia and Mazury in Olsztyn
Faculty of Mathematics and Computer Science

Field of Study: Computer Science

Igor Barcikowski

Application for Managing Reservations

Engineer's Thesis is performed
in the Department

under supervision of
supervisor

Olsztyn, 2026

Spis treści

Rozdział 1. Wstęp . . . 3
1.1. Motywacja . . . 3
1.2. Cel pracy . . . 3
1.3. Zakres Pracy . . . 3
1.4. Opis zawartości poszczególnych rozdziałów . . . 4
Rozdział 2. Teoria stanu wiedzy . . . 5
2.1. Przygotowanie teoretyczne . . . 5
2.2. Architektura aplikacji . . . 7
2.3. Analiza porównawcza . . . 9
Krótkie opisy serwisów . . . 9
Cechy wspólne (podobieństwa) . . . 10
Różnice . . . 10
Rozdział 3. Projekt i implementacja aplikacji . . . 11
3.1. Wykorzystane technologie i narzędzia . . . 11
3.2. Architektura aplikacji . . . 11
3.3. Wymagania funkcjonalne . . . 12
3.4. Wymagania niefunkcjonalne . . . 13
3.5. Diagram encji . . . 14
3.6. Opis struktury bazy danych . . . 15
3.7. Uzasadnienie wyboru PostgreSQL . . . 16
3.8. Diagram przypadków użycia . . . 17
3.9. Scenariusze przypadków użycia . . . 19
3.9.1. Scenariusz rezerwowania usługi . . . 19
3.9.2. Scenariusz płatności online . . . 19
3.9.3. Scenariusz tworzenia produktu . . . 20
Rozdział 4. Implementacja i dokumentacja techniczna . . . 21
4.1. Diagram komponentów . . . 21
4.1.1. Diagram komponentów . . . 22
4.1.2. Przeglądarka/Frontend . . . 22
4.1.3. Backend – Spring Security . . . 22
4.1.4. Backend - Kontrolery REST . . . 23
4.2. Diagramy sekwencyjne . . . 26
Rozdział 5. Testy i wyniki . . . 34
5.1. Testowanie . . . 34
5.1.1. Testy jednostkowe . . . 35
5.1.2. Testy integracyjne . . . 35
5.2. Nominatim i proces geokodowania . . . 37
5.3. Detale techniczne aplikacji . . . 40
5.3.1. JpaSpecificationExecutor . . . 40

5.3.2. Zabezpieczenia . . . 41
5.3.3. Tabela kluczowych endpointów . . . 42
Rozdział 6. Instrukcja obsługi . . . 43
6.1. Instrukcja uruchomienia aplikacji . . . 43
6.1.1. Wymagania wstępne . . . 43
6.1.2. Pobranie kodu źródłowego . . . 43
6.1.3. Konfiguracja zmiennych środowiskowych . . . 43
6.1.4. Środowisko VS Code (aplikacja kliencka) . . . 45
6.1.5. Uruchomienie . . . 47
6.1.6. Testowanie płatności . . . 47
6.1.7. Zatrzymanie aplikacji . . . 47
6.1.8. Uwagi dotyczące bezpieczeństwa . . . 47
Rozdział 7. Frontend . . . 48
7.1. Technologia . . . 48
7.2. Struktura projektu . . . 48
7.3. Ochrona tras . . . 49
7.4. Kalendarz i lista slotów . . . 49
7.4.1. Siatka miesiąca . . . 49
7.4.2. Dostępność dni . . . 49
7.4.3. Generowanie slotów . . . 50
7.4.4. Zajęte terminy i wybór slotu . . . 50
7.4.5. Ograniczenia . . . 50
7.5. Pozostałe elementy interfejsu . . . 50
7.6. Zrzuty ekranu UI . . . 51
Rozdział 8. Zakończenie . . . 59
8.1. Podsumowanie . . . 59
8.2. Wnioski . . . 59
8.3. Ograniczenia i możliwości rozwoju aplikacji . . . 60
Bibliografia . . . 61
Streszczenie . . . 63
Abstract . . . 64

Rozdział 1

Wstęp

1.1. Motywacja

Niniejsza aplikacja stara się odpowiedzieć na stale rosnące zapotrzebowanie na serwisy rezerwacyjne on-line. Serwisy te są już nieodłączną cechą naszego życia. Wraz z rozwojem technologii oraz języków programowania stają się one raz za razem coraz bardziej rozbudowane i nowoczesne. Począwszy od lokalnego lokalu fryzjerskiego do multi-miliardowej korporacji np. Booking, praktycznie każda z tych firm posiada własny serwis ułatwiający klientom zarezerwowanie np. usługi, noclegu, miejsc w kinie itd. Aplikacja skupia się na tym, aby usługodawcy nie musieli tworzyć swojej własnej strony internetowej, w celu zareklamowania się i umożliwienia rezerwacji ich usług klientom, lecz po założeniu konta w Aplikacji mogliby stworzyć swoje własne ogłoszenie wraz z opisem, datami dostępności, płatnościami itd. co pochłania wielokrotnie mniej czasu i funduszy niż stworzenie własnego serwisu internetowego.

1.2. Cel pracy

Aplikacja ma za zadanie umożliwić usługodawcom i usługobiorcom szybki i prosty przegląd i rezerwacje usług. Wiele aplikacji webowych skupiających się na rezerwacjach jest ściśle skoncentrowanych na usługach lub ogłoszeniach z tylko jednej dziedziny (np. noclegi, fryzjerstwo, salon kosmetyczny). Ta aplikacja stara się zapewnić jak największą uniwersalność podczas tworzenia ogłoszeń, zrzeszając szeroką gamę usługobiorców z różnych branż. Takie podejście ułatwi użytkownikom szukanie tego czego potrzebują ograniczając liczbę serwisów internetowych, które muszą znać i opanować.

1.3. Zakres Pracy

W ramach aplikacji zaimplementowano podejście architektury REST API w frameworku Spring Boot obsługujący logikę biznesową. Aplikacja kliencka w React komunikuje się z API. Do przechowywania danych o użytkownikach, ogłoszeniach i rezerwacjach wykorzystano relacyjną bazę danych PostgreSQL. Obsługa płatności została zrealizowana w testowym trybie (Stripe sandbox).

Aplikacja nie zaimplementowała następujących rzeczy: aplikacji mobilnej, tłumaczenie interfejsu na wiele języków (domyślnym językiem jest polski).

1.4. Opis zawartości poszczególnych rozdziałów

— W rozdziale 1. opisano motywację, cel i zakres pracy.
— W rozdziale 2. przedstawiono przygotowanie teoretyczne (system rezerwacyjny, architektura klient-serwer, bazy danych, frameworki) oraz analizę porównawczą istniejących serwisów.
— W rozdziale 3. opisano projekt systemu: technologie, wymagania, strukturę bazy danych, przypadki użycia i scenariusze.
— W rozdziale 4. przedstawiono implementację i dokumentację techniczną: diagram komponentów, backend oraz diagramy sekwencyjne procesów rezerwacji, płatności i logowania.
— W rozdziale 5. opisano testy jednostkowe i integracyjne wraz z wynikami, proces geokodowania z użyciem Nominatim oraz detale techniczne aplikacji (JpaSpecificationExecutor, zabezpieczenia, kluczowe endpointy).
— W rozdziale 6. zawarto instrukcję uruchomienia aplikacji.
— W rozdziale 7. opisano frontend oraz interfejs użytkownika wraz ze zrzutami ekranu.
— W rozdziale 8. zawarto podsumowanie, wnioski, ograniczenia i kierunki rozwoju.

Rozdział 2

Teoria stanu wiedzy

2.1. Przygotowanie teoretyczne

System rezerwacyjny to narzędzie - najczęściej w formie aplikacji lub platformy online, które umożliwia klientom samodzielne dokonywanie rezerwacji usług, terminów, lub zasobów. Użytkownikom najczęściej jest przedstawiana strona główna na której widnieje szeroka lista usług i praktycznie zawsze widniejąca razem z nią gama filtrów takich jak: wyszukiwanie po nazwie usługi, cena maksymalna/minimalna usługi, termin, lokalizacja. Filtry pomagają dopasować usługę do wymagań klienta. Serwis powinien również mieć obsługę płatności. W świecie który dąży do coraz większej cyfryzacji, płatności online stały się, można powiedzieć obowiązkową częścią każdego serwisu rezerwacyjnego. Nie oznacza to jednak że wszystkie aplikacje odstąpiły od płatności gotówkowych, niektóre serwisy oferują opcjonalną płatność gotówkową jeśli właściciel ogłoszenia woli taką drogę opłaty. Może to otworzyć drogę do większej ilości klientów, którzy wolą płatności gotówkowe. Z punktu widzenia posiadacza biznesu, stworzenie własnej strony do obsługi rezerwacji w firmie, lub właśnie opublikowanie ogłoszenia na portalu zajmującego się ogłoszeniami, może dać wiele korzyści. Klienci mogą dokonać rezerwacji o każdej porze dnia. Niewygodna i czasochłonna mogłaby też być sytuacja gdy klienci w celu dokonania rezerwacji musieliby się kontaktować telefonicznie lub pojawić się osobiście w lokalu. Co za tym idzie pojawia się problem płatności, w wyniku braku systemu który pozwalałby na opłacenie wizyty online, usługodawca musiałby się liczyć z tym, że niektórzy klienci mogliby nie przyjść na wizytę, co blokowałoby innych klientów którzy mogliby zarezerwować wizytę w tym czasie. Kolejną korzyścią może być wzrost ilości klientów. Zwiększając ekspozycję swojego lokalu/ogłoszenia, mogłoby to prowadzić do większej ilości klientów którzy dowiedzieliby się o ofercie. Daje to też możliwości wypromowania oferty poprzez płatne reklamy internetowe które często są tańsze i przyciągają większą gamę klientów niż fizyczne billboardy.

Warto omówić z jakimi problemami zmierzają się twórcy takich serwisów. Pierwszą z nich mogą być właśnie płatności, w szczególności jeśli aplikacja skupia się na pozyskaniu klientów z wielu krajów. W takich sytuacjach ilość dróg płatności często jest ograniczona do zaledwie kilku jeśli nie tylko jednej. Najczęściej spotykaną drogą w takich przypadkach jest droga płatności kartą, lub przelewem. Są to metody które większość społeczeństw używa więc problem międzynarodowej integralności jest mniejszy. Sytuacja może być inna jeśli serwis jest kierowany do klientów jednego kraju. Wtedy łatwiej jest twórcom zdecydować jakie metody płatności mają być obsługiwane w serwisie. BLIK, karty płatnicze i przelewy online pozostają najpopularniejszymi metodami płatności internetowych wśród polskich konsumentów [16].

Kolejnym problemem dla niektórych serwisów jest stworzenie aplikacji która będzie miała potrzebne funkcjonalności a zarazem nie będzie przesadnio skomplikowana, aby klient był w stanie intuicyjnie wiedzieć jak prosto i szybko znaleźć odpowiednią usługę i dokonać rezerwacji. Trzeba wziąć pod uwagę również różnice w umiejętnościach technicznych potencjalnych użytkowników, co zwiększa potrzebę stworzenia intuicyjnego interfejsu. Rozwiązaniem może być umieszczenie poradników, infolinii lub nowoczesnego czatu AI wytrenowanego w wiedzy o aplikacji.

Trzecim problemem może się okazać fakturowanie. A konkretnie integracja aplikacji z usługodawcą, a od pewnego czasu w Polsce również z systemem KSeF. Obowiązek wystawiania faktur ustrukturyzowanych przez KSeF obejmie od 1 kwietnia 2026 roku pozostałych przedsiębiorców [17]. Wiele serwisów szczególnie tych z międzynarodową gamą usługodawców i klientów np. Booking, rozwiązuje ten problem mówiąc jawnie: "Fakturami zajmuje się bezpośrednio dostawca usług. Serwis Booking.com for Business nie wystawia faktur, ale szczegółowy wykaz kosztów oraz potwierdzenie płatności znajdziesz w potwierdzeniu rezerwacji".[15] Jest to jak na razie standard na rynku.

2.2. Architektura aplikacji

Najpowszechniejszą architekturą aplikacji webowych jest architektura klient-serwer. Komunikacja między tymi warstwami odbywa się zazwyczaj zgodnie z konwencją REST API [19]. Jest to model w którym system dzieli się na dwie części: klient i serwer.

Warstwa klienta (inaczej frontend) bezpośrednio wchodzi w interakcje z użytkownikiem, prościej mówiąc jest to to co użytkownik widzi czyli inaczej interfejs wraz z danymi. Klient nie jest tylko odpowiedzialny za wyświetlanie interfejsu, lecz również za przechwytywanie akcji użytkownika, gdy ten np. kliknie przycisk "rezerwuj". Następnie często ta akcja jest przekazywana do naszego serwera w typie danych JSON. Klient nie ma bezpośredniego dostępu do logiki biznesowej ani bazy danych, dzięki temu użytkownik, który chciałby dostać się do naszego serwera, lub bazy danych ma zdecydowanie utrudnione zadanie do zrobienia tego prosto z przeglądarki. Każda metoda logiki biznesowej po stronie serwera ma przypisaną unikalną kombinacje metody HTTP i URL. W taki sposób klient wysyła żądanie, aby serwer wykonał odpowiednią akcję.

Serwer (inaczej backend) jest odpowiedzialny za przechowywanie danych, logiki biznesowej i otrzymywanie informacji o akcjach użytkownika od klienta. Serwer łączy się bezpośrednio z bazą danych i na podstawie otrzymanych informacji z frontendu - po sprawdzeniu wszystkich wymogów, zabezpieczeń, wykonuje odpowiednią logikę biznesową, często zmieniając stan rzeczy bazy danych, wysyła maile, nadaje dostępność użytkownikowi i wiele innych.

Taki podział nie jest przypadkowy, robi się to ze względu na kilka rzeczy. Pierwszym powodem jest bezpieczeństwo danych. W przypadku gdy logika biznesowa naszej aplikacji znajdowała się po stronie klienta, mogłoby się zdarzyć, że poprzez edycję kodu w przeglądarce, użytkownik mógłby zmienić tą logikę, przez co obejść określone zasady. Dzięki oddzieleniu logiki biznesowej od warstwy frontendowej, serwer nie musi wiedzieć, jaki rodzaj klienta wysyła żądanie - może to być przeglądarka, aplikacja mobilna czy inny system. Komunikacja ta odbywa się najczęściej za pośrednictwem protokołu HTTP: klient wysyła zapytanie, a serwer odsyła odpowiedź [20]. Taki sposób komunikacji sprawia, że ten sam serwer może jednocześnie obsługiwać wiele różnych typów klientów, bez konieczności tworzenia osobnej logiki dla każdego z nich. Kolejnym ważnym aspektem jest skalowalność. Separacja pozwala na zwiększenie przepustowości serwera poprzez np. dodanie instancji, bez duplikowania kolejnego klienta. Jeszcze jednym ważnym atutem jest ułatwienie testowalności. Podczas wykonywania testów jednostkowych, można skupić się jedynie na testowaniu np. logiki biznesowej nie musząc do testu angażować bazy danych ani nie sprawdzania zwracanych statusów HTTP.

W systemach spotykamy się z kilkoma typami baz danych takim jak:

— Relacyjne (SQL). Dane przedstawione w tabelach, zawierają unikalny identyfikator zwany kluczem głównym. Relacje pomiędzy tabelami definiują klucze obce które są odwołaniem do klucza głównego powiązanej tabeli. To sprawia, że te bazy danych najlepiej sprawdzają się w aplikacjach właśnie z dużą ilością powiązań między tabelami, oraz gdy potrzeba dużej swobody w tworzeniu zapytań do bazy danych.

— Nierelacyjne (NoSQL). Obejmują kilka różnych podejść do przechowywania danych, m.in. bazy dokumentowe, klucz-wartość, kolumnowe czy grafowe. Jednym z nich są bazy dokumentowe, w których dane zapisywane są w postaci dokumentów, najczęściej w formacie JSON. Każdy dokument posiada unikalny identyfikator, ale nie istnieją między nimi tradycyjne powiązania - dane, które w bazie relacyjnej znajdowałyby się w osobnych, powiązanych tabelach, tutaj często muszą zostać zapisane razem, w jednym dokumencie zagnieżdżając dane w konkretnych polach. Sprawdzają się dobrze, gdy struktura bazy danych nie wymaga wielu złożonych powiązań ani zróżnicowanych zapytań, ponieważ dopasowanie struktury dokumentu do każdego nowego rodzaju zapytania może przy dużej ilości danych znacząco zwiększać zajmowaną przestrzeń dyskową.

Framework - szkielet do budowy aplikacji. Definiuje on strukturę aplikacji oraz ogólny mechanizm jej działania, a także dostarcza zestaw komponentów i bibliotek ogólnego przeznaczenia do wykonywania określonych zadań. Programista tworzy aplikację, rozbudowując i dostosowując poszczególne komponenty do wymagań realizowanego projektu, tworząc w ten sposób gotową aplikację. [18] Jest to zbiór funkcjonalności oraz bibliotek która nie tylko przyspiesza i ułatwia programowanie aplikacji, ale również narzuca z góry wzorce.

Poniżej widnieje kilka przykładów najpopularniejszych frameworków frontendowych [21]: **React** (JavaScript/TypeScript) Wykorzystywany do budowy dynamicznych interfejsów użytkownika, takich jak panele administracyjne i platformy ogłoszeniowe. Na jego bazie powstał **Next.js**, rozszerzający możliwości Reacta m.in. o renderowanie po stronie serwera, co poprawia optymalizację pod kątem SEO[1] oraz wydajność ładowania strony. Innym powszechnie stosowanym rozwiązaniem jest **Angular**, framework oparty w całości na TypeScript, rozwijany przez Google, często wybierany w dużych aplikacjach skupionych na długoterminowy rozwój ze względu na narzuconą, spójną strukturę projektu.

Poniżej widnieje kilka przykładów najpopularniejszych frameworków backendowych [22]: **Django**, framework napisany w języku Python, dobrze sprawdzający się w aplikacjach z wysokim użyciem bazy danych, dzięki wbudowanemu mechanizmowi ORM[2] oraz panelowi administracyjnemu, który generuje się automatycznie. **FastAPI**, również w Pythonie, cechuje go wysoka wydajność oraz automatyczne generowanie dokumentacji API w Swagger, co czyni go popularnym wyborem przy budowie nowoczesnych rozwiązań. **NestJS**, wykorzystujący TypeScript, występuje w nim spójna struktura inspirowana Angularem, co ułatwia organizację kodu w większych projektach backendowych. Z kolei **Spring Boot**, oparty na języku Java, jest szeroko stosowany w dużych systemach klasy enterprise[3], m.in. ze względu na dojrzałość ekosystemu oraz zaufanie i stabilność w środowiskach korporacyjnych. [22].

[1] SEO (ang. *Search Engine Optimization*) - zbiór działań mających na celu poprawę widoczności strony internetowej w wynikach wyszukiwania wyszukiwarek internetowych, takich jak Google.

[2] ORM (ang. *Object-Relational Mapping*) - mechanizm pozwalający na operowanie na danych z bazy za pomocą obiektów i metod danego języka programowania, bez konieczności pisania zapytań SQL bezpośrednio.

[3] Enterprise - określenie stosowane wobec systemów informatycznych o dużej skali oraz wysokich wymaganiach dotyczących niezawodności i bezpieczeństwa, typowych dla dużych organizacji i przedsiębiorstw.

2.3. Analiza porównawcza

W tej części przedstawiono trzy serwisy internetowe, które są inspiracją do tej aplikacji webowej. Są to jedne z liderów na rynku rezerwacji usług, obejmujących między innymi wizyty fryzjerskie, auto detailing, usługi programistyczne.

Krótkie opisy serwisów

Booksy:

Booksy to największa i najszybciej rozwijająca się platforma do rezerwacji usług beauty. Celem jest łączenie usługodawców z ich klientami oraz dostarczanie im narzędzi, których potrzebują, aby rozwijać swój biznes. Używa ją ponad 44 miliony użytkowników.

Strona domowa:

`https://booksy.com/`

Fiverr:

Fiverr jest internetową platformą pośredniczącą w nawiązywaniu współpracy pomiędzy freelancerami a klientami biznesowymi. Umożliwia osobom samozatrudnionym oferowanie swoich usług, natomiast przedsiębiorcom i właścicielom firm — wyszukiwanie wykonawców do realizacji zadań krótko- i długoterminowych. Firma zanotowała 3,1 miliona kupujących w 2025 roku.

Strona domowa:

`https://www.fiverr.com/`

Vagaro:

Internetowy kalendarz rezerwacji oferowany przez Vagaro umożliwia klientom przeglądanie dostępnych terminów oraz wybór najbardziej dogodnego czasu wizyty w salonach kosmetycznych, spa i obiektach fitness. Platforma jest skierowana bardziej na ściśle spersonalizowane systemy dla firm, choć są również dostępne domyślne konfiguracje jeśli firma nie potrzebuje własnej strony z niedomyślnymi funkcjonalnościami. Z platformy korzysta ponad 90 000 firm.

Strona domowa:

`https://www.vagaro.com/`

Cechy wspólne (podobieństwa)

— Wszystkie analizowane aplikacje działają w podobny sposób. Użytkownik wchodząc na stronę główną, widzi listę kategorii oraz ma możliwość wpisania konkretnej usługi, której szuka. Następnie otrzymuje listę ofert mieszczących się w wybranej kategorii lub odpowiadających podanym kryteriom wyszukiwania.
— We wszystkich przypadkach użytkownik po wybraniu oferty przechodzi do dedykowanego widoku konkretnego usługodawcy, gdzie może zapoznać się ze szczegółami oferty oraz zarezerwować wizytę/usługę.
— Wszystkie serwisy pełnią funkcję pośrednika pomiędzy usługodawcą a klientem. Ich zadaniem jest ułatwienie znalezienia odpowiedniej usługi oraz nawiązania kontaktu lub dokonania rezerwacji.

Różnice

— Na stronach Booksy i Vagaro użytkownik ma możliwość filtrowania ofert według lokalizacji oraz czasu wykonywanej usługi, ponieważ te serwisy są skoncentrowane głównie na rezerwacji usług stacjonarnych, takich jak fryzjerstwo, kosmetyka czy usługi wellness. Fiverr natomiast koncentruje się na usługach cyfrowych wykonywanych zdalnie, takich jak projektowanie stron, tworzenie materiałów reklamowych czy usługi programistyczne.
— Usługi oferowane w Booksy i Vagaro są często bardziej z góry określone, na przykład strzyżenie, masaż, manicure. W przypadku Fiverr oferta bywa bardziej elastyczna i indywidualna, bo wiele usług jest dopasowywanych do konkretnego zlecenia klienta.
— Istotną różnicą jest również sposób kontaktu i omówienia realizacji usługi pomiędzy klientem a usługodawcą. W przypadku Booksy i Vagaro, w większości przypadków klient nie musi w żaden sposób kontaktować się bezpośrednio z usługodawcą. Lecz w serwisie Fiverr bardzo często wymagane jest szczegółowe opisanie wymagań i potrzeb przez klienta, jak również ewentualne poprawki.

Rozdział 3

Projekt i implementacja aplikacji

3.1. Wykorzystane technologie i narzędzia

— Java — język programowania wykorzystany do implementacji części serwerowej.
— Spring Boot — framework użyty do budowy aplikacji backendowej.
— Spring Data JPA — narzędzie do komunikacji z bazą danych.
— Spring Security — odpowiedzialny za uwierzytelnianie i autoryzację użytkowników.
— PostgreSQL — relacyjna baza danych.
— JWT (JSON Web Token) — mechanizm tokenowego uwierzytelniania użytkowników.
— React — biblioteka JavaScript wykorzystana do stworzenia interfejsu użytkownika.
— TypeScript — statycznie typowany nadzbiór JavaScript wykorzystany w warstwie frontendowej.
— Vite — narzędzie do budowania i uruchamiania aplikacji frontendowej.
— React Router — biblioteka do obsługi routingu w aplikacji React.
— Leaflet i react-leaflet — biblioteki do wyświetlania interaktywnych map z lokalizacjami ofert.
— Nominatim (OpenStreetMap) — usługa geokodowania adresów wykorzystana przy dodawaniu produktów.
— Docker — narzędzie pomocnicze do uruchamiania środowiska aplikacji.
— Git i GitHub — narzędzia do kontroli wersji kodu źródłowego.
— Stripe — narzędzie do implementacji obsługi płatności (demo).
— SpringDoc OpenAPI (Swagger UI) – do generowania i testowania dokumentacji REST API
— Spring Boot mail - narzędzie do generowania i wysyłania maili

3.2. Architektura aplikacji

Aplikacja została zaprojektowana w architekturze klient-serwer. Część frontendowa, przygotowana w React, odpowiada za interakcję z użytkownikiem. Część backendowa, przygotowana w frameworku Spring Boot, udostępnia REST API odpowiedzialne za obsługę logiki biznesowej, komunikację z bazą danych oraz kontrolę dostępu do zasobów. Została zaimplementowana zgodnie ze wzorcem architektonicznym MVC (Model-View-Controller). Warstwa modelu odpowiada za reprezentację danych i dostęp do bazy danych, warstwa kontrolerów udostępnia endpointy REST API obsługujące żądania klienta, a rolę warstwy widoku pełni aplikacja kliencka napisana w React, która prezentuje dane zwrócone przez API. Takie podejście pozwala na oddzielenie warstw aplikacji, przez co pliki z kodem są uporządkowane, co wpływa na łatwiejszą nawigację w przypadku, gdy programista szuka konkretnego fragmentu aplikacji, np. gdyby nastała potrzeba zmiany logiki na poziomie bazy danych, byłoby wiadomo, że znajduje się ona na poziomie modelu. Ponadto rozdzielenie różniących się od siebie aspektów aplikacji, powoduje że gdy jest potrzeba zmiany tylko logiki bazy danych, nie ma potrzeby zmieniania kodu w innych warstwach aplikacji, ponieważ nie są one ściśle powiązane w jednej klasie.

3.3. Wymagania funkcjonalne

Do podstawowych wymagań funkcjonalnych aplikacji należą:
— rejestracja i logowanie użytkowników,
— rozróżnienie ról użytkowników, na przykład klient i usługodawca,
— dodawanie, edycja i usuwanie ofert usług,
— przeglądanie listy dostępnych usług,
— wyszukiwanie i filtrowanie ofert,
— wyświetlanie szczegółów wybranej oferty,
— tworzenie rezerwacji przez klienta,
— zarządzanie własnymi ofertami przez usługodawcę,
— zarządzanie płatnościami oraz fakturami,
— możliwość zobaczenia dostępnych ofert na mapie,
— otrzymywanie maili przypominających.

3.4. Wymagania niefunkcjonalne

Przechowywanie haseł Hasła użytkowników są przechowywane wyłącznie w postaci skrótu kryptograficznego (BCrypt). Jest to funkcja jednokierunkowa, co oznacza, że z zapisanego skrótu nie da się odtworzyć oryginalnego hasła. Algorytm używa również tzw. sól, czyli losowy ciąg znaków który jest dodawany do hasła przed obliczeniem hasha. Dzięki temu te same hasła dwóch różnych użytkowników będą różne po wykonaniu skrótu. Podczas logowania wpisane hasło jest haszowane w ten sam sposób i porównywane z zapisanym skrótem w tabeli.

Uwierzytelnianie i autoryzacja Dostęp do chronionych zasobów wymaga uwierzytelnienia za pomocą tokenu JWT, a uprawnienia są przydzielane na podstawie ról (USER, PROVIDER, ADMIN). Token jest podpisywany algorytmem HS256 z użyciem klucza, który jest tajny i przechowywany wyłącznie na serwerze, dzięki czemu nie można go zmienić, co mogłoby poskutkować zmianą przypisanej roli (np. z USER do ADMIN) przez osobę, która nie powinna była móc tego zrobić. Token JWT wygasa automatycznie po 30 minutach, co zmniejsza czas, w którym nieupoważniona osoba, która przechwyciłaby ważny token, mogłaby wysyłać żądania w imieniu użytkownika.

Bezpieczeństwo płatności Dane kart płatniczych nie są przetwarzane ani przechowywane w systemie - obsługę płatności realizuje zewnętrzny operator Stripe w środowisku testowym. Używana jest po stronie backendu dedykowana biblioteka `stripe-java`, która pozwala m.in. na otrzymywanie Webhooków [1] z Stripe, potwierdzania ich oraz potwierdzenia, że pochodzą właśnie od Stripe. Webhooki rozwiązują również problem podwójnych rezerwacji, ponieważ dopiero kiedy serwer otrzyma Webhook potwierdzający płatność, w bazie danych po stronie serwera rezerwacja zmieni status na "CONFIRMED", w przypadku niepotwierdzenia płatności - rezerwacja usunie się po 15 minutach przez cykliczną metodę.

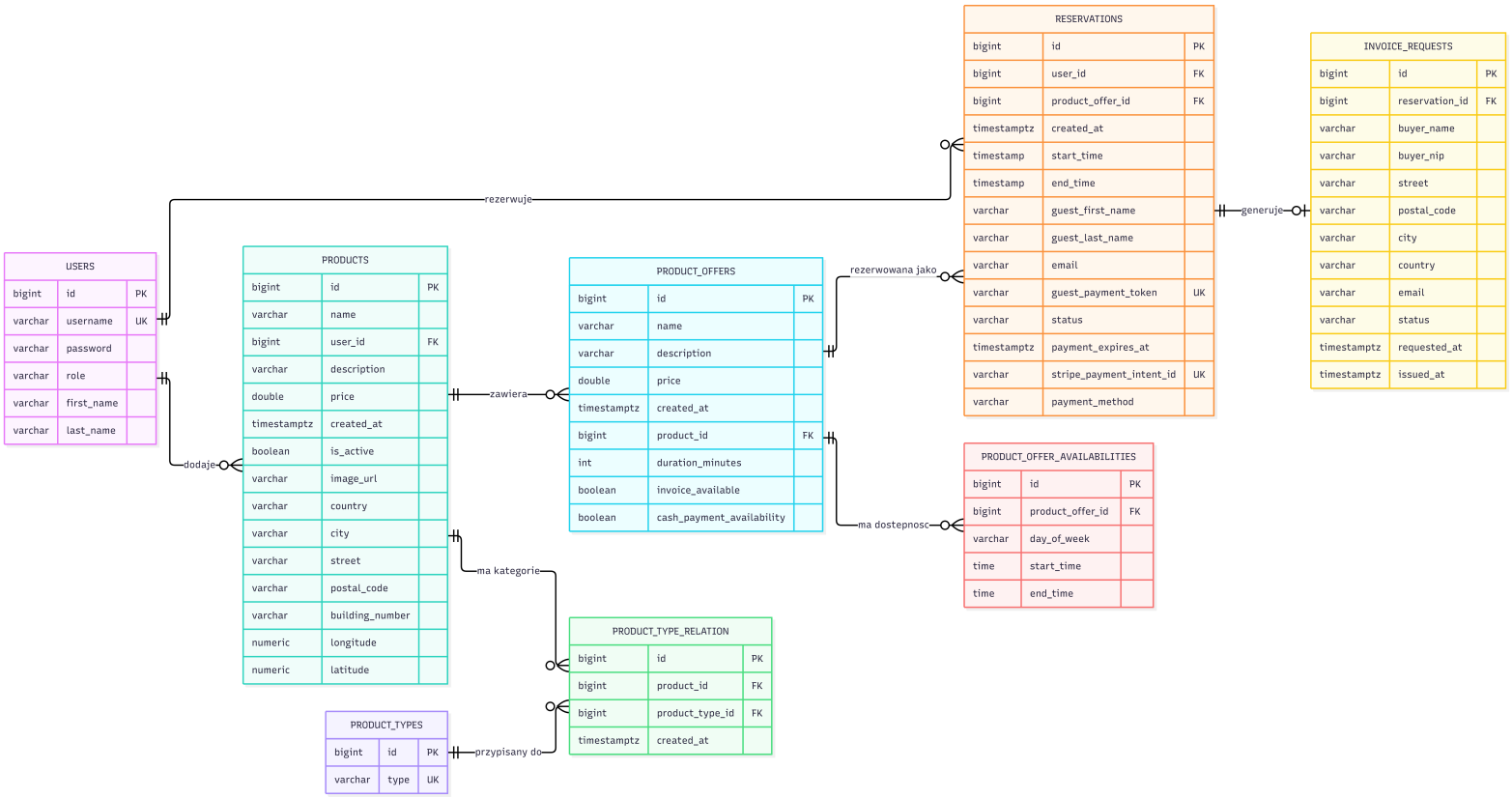
Wydajność Czas odpowiedzi API dla typowych operacji nie przekracza 1 sekundy przy standardowym obciążeniu.

Skalowalność Bezstanowa architektura API (JWT) umożliwia skalowanie poziome poprzez uruchomienie wielu instancji serwera. Aplikacja posiada również dokumentację generowaną w Swagger, co może zdecydowanie ułatwić dalsze programowanie oraz skalowalność.

Responsywność interfejsu Interfejs użytkownika działa poprawnie na komputerach oraz urządzeniach mobilnych. Po stronie klienta są używane zapytania o media (ang. *media queries*), które automatycznie dostosowują wygląd strony do wielkości ekranu na którym jest wyświetlana.

[1] Webhooki są mechanizmem wykorzystywanym w tworzeniu stron internetowych w celu umożliwienia komunikacji w czasie rzeczywistym i wymiany danych między różnymi systemami lub aplikacjami. Umożliwiają one jednej aplikacji powiadomienie innej aplikacji o określonym zdarzeniu lub wyzwalaczu, który wystąpił.

3.5. Diagram encji



Rysunek 3.1: Diagram encji bazy danych aplikacji

3.6. Opis struktury bazy danych

users — przechowuje dane kont użytkowników. Pole `username` jest unikalne, co uniemożliwia rejestrację dwóch kont o tej samej nazwie użytkownika. Dodatkowo tabela zawiera pola `password` (hasz hasła). Hasła użytkowników nigdy nie są przechowywane w bazie danych w postaci jawnej. Podczas rejestracji (metoda `register`) hasło przekazane przez użytkownika jest haszowane za pomocą algorytmu `BCrypt`, a dopiero tak przetworzony hasz zapisywany. `role` (określające, czy użytkownik jest klientem czy usługodawcą) oraz `first_name` i `last_name`.

products — reprezentuje ogłoszenia dodawane przez usługodawców. Zawiera klucz obcy `user_id`, wskazujący na właściciela ogłoszenia. Poza podstawowymi polami opisowymi (`name`, `description`, `price`, `image_url`) tabela przechowuje pełne dane adresowe (`country`, `city`, `street`, `postal_code`, `building_number`) oraz współrzędne geograficzne (`longitude`, `latitude`), wykorzystywane do wyświetlania ogłoszeń na mapie.

product_types — słownikowa tabela typów/kategorii usług. Pole `type` jest unikalne, dzięki czemu każda kategoria występuje w systemie tylko raz.

product_type_relation — tabela pośrednicząca realizująca relację wiele-do-wielu pomiędzy `products` i `product_types`. Zawiera dwa klucze obce (`product_id`, `product_type_id`) oraz złożone ograniczenie unikalności na parze tych kolumn, co zapobiega wielokrotnemu przypisaniu tego samego typu do tego samego ogłoszenia.

product_offers — różne warianty danego produktu. Zawiera klucz obcy `product_id` oraz pola `duration_minutes` (czas trwania usługi), `invoice_available` (czy usługodawca umożliwia wystawienie faktury) i `cash_payment_availability` (czy dopuszczalna jest płatność gotówkowa).

product_offer_availabilities — definiuje dostępne terminy dla danej oferty. Zawiera klucz obcy `product_offer_id`, pole `day_of_week` oraz godziny `start_time`/`end_time`. Na parze (`product_offer_id`, `day_of_week`) nałożone jest ograniczenie unikalności, uniemożliwiające zdefiniowanie dwóch sprzecznych okien dostępności w tym samym dniu.

reservations — Tabela przechowująca rezerwacje. Zawiera dwa klucze obce: `user_id` (nullable — pozwala na rezerwacje bez zalogowania) oraz `product_offer_id`. Dla rezerwacji dokonywanych przez gości istnieją również pola: `guest_first_name`, `guest_last_name` oraz unikalny `guest_payment_token`, służący do identyfikacji płatności bez konieczności logowania. Pole `stripe_payment_intent_id` (unikalne) wiąże rezerwację z konkretną transakcją w systemie Stripe, a `status` i `payment_expires_at` obsługują cykl życia płatności (oczekująca, opłacona, wygasła).

invoice_requests — żądania wystawienia faktury, powiązane z konkretną rezerwacją relacją jeden-do-jednego: klucz obcy `reservation_id` jest unikalna, więc do jednej rezerwacji może zostać przypisane maksymalnie jedno żądanie faktury. Tabela przechowuje dane nabywcy (`buyer_name`, `buyer_nip`, adres) potrzebne do wystawienia faktury.

3.7. Uzasadnienie wyboru PostgreSQL

PostgreSQL jest bazą relacyjną o stale rozbudowywanej infrastrukturze oraz dużej ilości bibliotek i narzędzi wspierających programowanie, np. `JpaSpecificationExecutor` ze Spring Data JPA, umożliwiającym budowanie dynamicznych zapytań filtrujących bez konieczności ręcznego pisania wielu wariantów zapytań SQL.

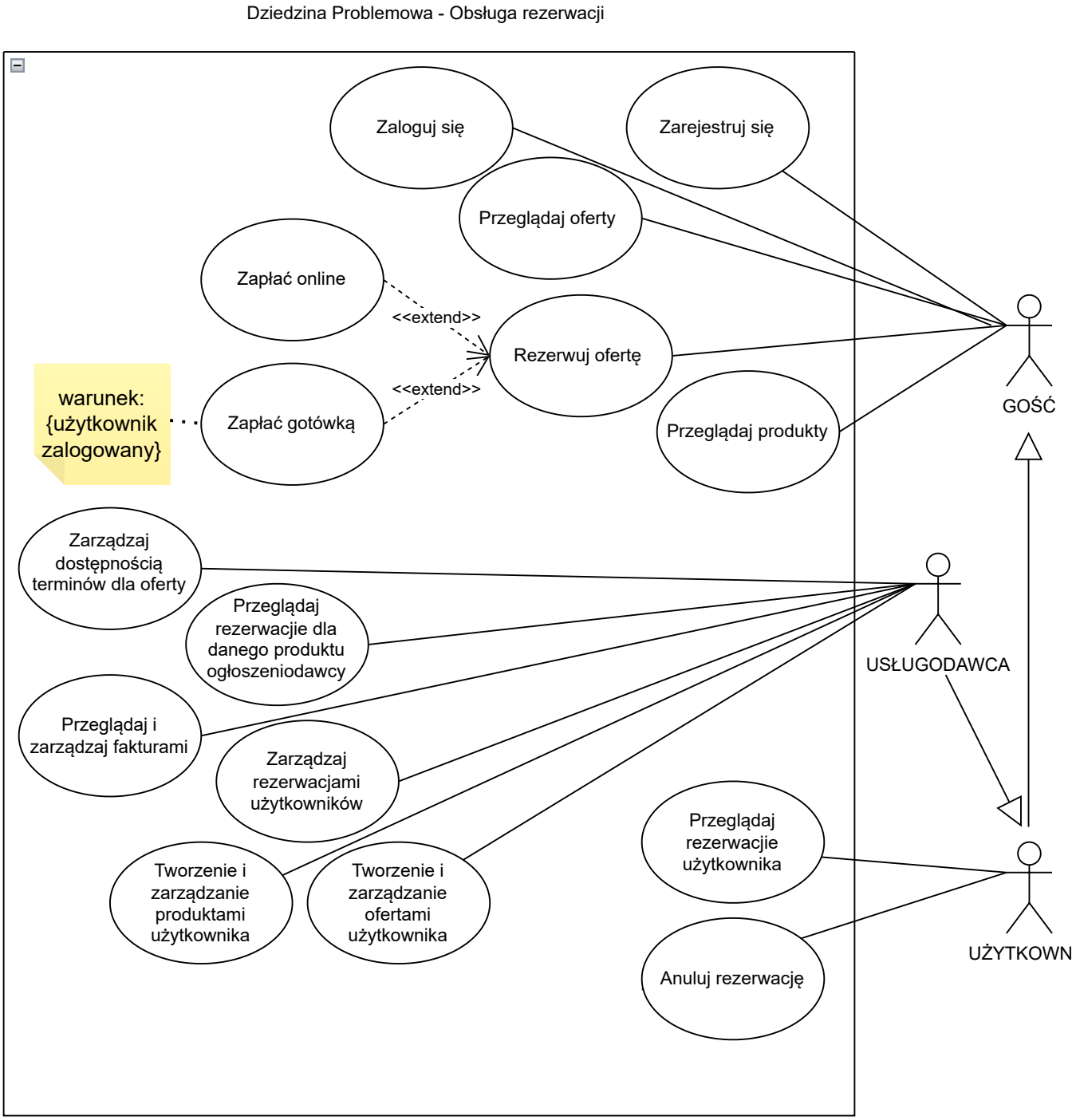
Relacyjne bazy danych pozwalają na definiowanie wielu złożonych relacji pomiędzy tabelami - wystarczy określić je za pomocą kluczy obcych w SQL. Kolejną zaletą ściśle zdefiniowanych struktur tabel jest integralność danych: baza sama pilnuje, aby rekord w tabeli referencyjnej odwoływał się do powiązanego wiersza w tabeli nadrzędnej, co eliminuje ryzyko niespójności. Dzięki temu unikamy tzw. osieroconych danych, czyli, gdy w tabeli nadrzędnej rekord zostaje usunięty, baza automatycznie potrafi rekord w bazie podrzędnej usunąć. W bazach nierelacyjnych (np. dokumentowych, NoSQL), gdzie dane bywają celowo zdenormalizowane, dane ze sobą powiązane, lub te same dane mogą występować w wielu miejscach jednocześnie, co wymaga ręcznego pilnowania spójności przy każdej aktualizacji.

Powoduje to również mniejsze zużycie zasobów bazy danych przy zapisywaniu nowych rekordów. Jednakże często podczas zapytań zawierających wiele złączeń (`JOIN`) pomiędzy tabelami - odczyt danych może być zdecydowanie wolniejszy, oraz bardziej kosztowny obliczeniowo. PostgreSQL używa inteligentego planera zapytań i indeksów co zmniejsza znacznie koszt zapytań.

Warto rozważyć hipotetyczną sytuację, w której aplikacja wymagałaby wielu zapytań o głębokim łańcuchu relacji (czyli wielopoziomowe zapytanie przechodzi po wielu tabelach kolejno żeby dostać się do potrzebnych informacji), w tym przypadku można byłoby się zastanowić nad wprowadzeniem dodatkowej bazy grafowej, np. Neo4j obok głównej bazy relacyjnej. Jest to podejście znane jako *polyglot persistence*, gdzie każda technologia odpowiada za wybrane zapytania, które są dla niego najlepiej dopasowane. Przyspieszyłoby to działanie pewnych aspektów aplikacji, ale oczywiście mierzyłoby się to z dodatkowymi kosztami utrzymywania architektury oraz zwiększyłoby to złożoność architektury samej aplikacji. Dlatego decyzja o wprowadzeniu tego modelu powinna zależeć od częstotliwości wywoływania tych złożonych zapytań.

Biorąc pod uwagę powyższe czynniki, w aplikacji ostatecznie zastosowano relacyjną bazę danych PostgreSQL, ponieważ jej model danych i silnie powiązane ze sobą tabele takie jak: oferty, rezerwacje, kategorie itd. — naturalnie wpasowuje się w ten właśnie wybór, a częstotliwość głębokich zapytań na wielu tabelach na raz w aplikacji nie uzasadnia wprowadzania dodatkowej bazy grafowej.

3.8. Diagram przypadków użycia



Rysunek 3.2: Diagram przypadków użycia systemu rezerwacji usług

Na powyższym diagramie przypadków użycia, przedstawione zostały wszystkie funkcjonalności, wraz z aktorami, które reprezentują role występujące w aplikacji. Aktorzy są połączeni strzałkami, takie podejście ma na celu zobrazowanie dziedziczenia uprawnień pomiędzy nimi. USŁUGODAWCA (który w kodzie aplikacji jest zapisany jako PROVIDER z angielskiego), ma najwięcej uprawnień z całej trójki. Wszystkie uprawnienia, które posiada UŻYTKOWNIK oraz GOŚĆ - USŁUGODAWCA również posiada. Użytkownik staje się usługodawcą, gdy zostaną mu nadane uprawnienia przez operatora systemu z dostępem do bazy danych.

Rolą która nie posiada uprawnień, które usługodawca posiada jest UŻYTKOWNIK (USER), czyli osoba, która jest zalogowana w aplikacji. Ma on dostęp do takich funkcjonalności jak: Przegląd ofert, filtrowanie ofert, rezerwacja oferty z możliwością płatności na miejscu, dostęp do panelu użytkownika, w którym może śledzić stan i informacje o jego rezerwacjach, oraz zarządzać tymi rezerwacjami (anulować, dokończyć płatność, filtrować). Nie posiada jednak funkcji dostępnych dla usługodawców takich jak: tworzenie i zarządzanie ofertami, zarządzanie rezerwacjami na daną ofertę, zarządzanie fakturami.

Kolejną i ostatnią rolą o najmniejszej ilości uprawnień jest GOŚĆ. Jest to osoba która nie jest zalogowana w aplikacji. W skład dozwolonych operacji gościa są: przeszukiwanie i filtrowanie ofert, rezerwowanie ofert (wyłącznie płatność online). Przy rezerwacji, gość automatycznie dostaje maila z linkiem do wznowienia płatności (z okresem ważności 15 minut), w przypadku gdyby gość niechcąco wyszedł z aplikacji. Wtedy rezerwacja nigdy nie byłaby ważna, ponieważ nie została potwierdzona płatność. Takiego mechanizmu wysłania maila ze wznowieniem płatności nie ma dla zalogowanych użytkowników, ponieważ można to zrobić z panelu użytkownika.

3.9. Scenariusze przypadków użycia

3.9.1. Scenariusz rezerwowania usługi

SYTUACJA: Aby skorzystać z usługi, np. u fryzjera czy masażysty, klient zazwyczaj musi zadzwonić lub napisać do usługodawcy i wspólnie ustalić wolny termin. Zajmuje to czas obu stronom i często jest niemożliwe poza godzinami pracy lokalu.

OPIS: Anna chce umówić się na strzyżenie. Zamiast dzwonić do kilku salonów, wchodzi do aplikacji, filtruje usługi po swoim mieście i typie usługi, wybiera salon, a następnie ofertę „strzyżenie damskie". W kalendarzu widzi wolne terminy, wybiera ten, który jej pasuje, i potwierdza rezerwację.

Specyfikacja:
1. Klient wyszukuje i filtruje dostępne produkty (np. po mieście i typie usługi)
2. Klient wybiera produkt i jedną z jego ofert
3. Klient wybiera wolny termin z kalendarza dostępności oferty
4. Klient wybiera metodę płatności (płatność na miejscu dostępna tylko dla zalogowanych użytkowników i tylko jeśli oferta ją dopuszcza)
5. System sprawdza, czy wybrany termin nie koliduje z inną rezerwacją
6. System zapisuje rezerwację lub informuje klienta, że termin jest już zajęty

3.9.2. Scenariusz płatności online

SYTUACJA Usługodawcy często mają problem z klientami, którzy rezerwują termin, a potem się nie pojawiają. Pobieranie zaliczek przelewem wymaga z kolei ręcznego sprawdzania, czy wpłata dotarła.

OPIS: Marek, nie zakładając konta, rezerwuje masaż i wybiera płatność online. W trakcie wpisywania danych karty przypadkowo zamyka kartę przeglądarki. Na swojej skrzynce znajduje jednak wiadomość z linkiem do wznowienia płatności, dzięki któremu kończy ją w ciągu kilku minut, a jego rezerwacja zostaje potwierdzona.

Specyfikacja:
1. Klient po złożeniu rezerwacji zostaje przekierowany do formularza płatności
2. System tworzy płatność w serwisie Stripe
3. Klient podaje dane karty i zatwierdza płatność
4. Stripe powiadamia system o udanej płatności (webhook)
5. System zmienia status rezerwacji na opłaconą
6. W przypadku przerwania płatności gość może ją wznowić przez link z wiadomości e-mail (ważny 15 minut), a zalogowany użytkownik z poziomu panelu użytkownika

3.9.3. Scenariusz tworzenia produktu

SYTUACJA: Małe firmy usługowe często nie mają własnej strony z systemem rezerwacji. Informacje o cenach, usługach i godzinach pracy są rozproszone po mediach społecznościowych, a zapisy odbywają się telefonicznie.

OPIS: Kasia prowadzi salon kosmetyczny i posiada konto usługodawcy. Dodaje w aplikacji swój salon wraz z opisem, adresem i zdjęciami. Następnie tworzy oferty, np. „manicure hybrydowy" trwający 60 minut z możliwością płatności na miejscu, i ustala, że przyjmuje klientów od poniedziałku do piątku w godzinach 9–17. Od tej chwili klienci mogą samodzielnie rezerwować u niej wizyty.

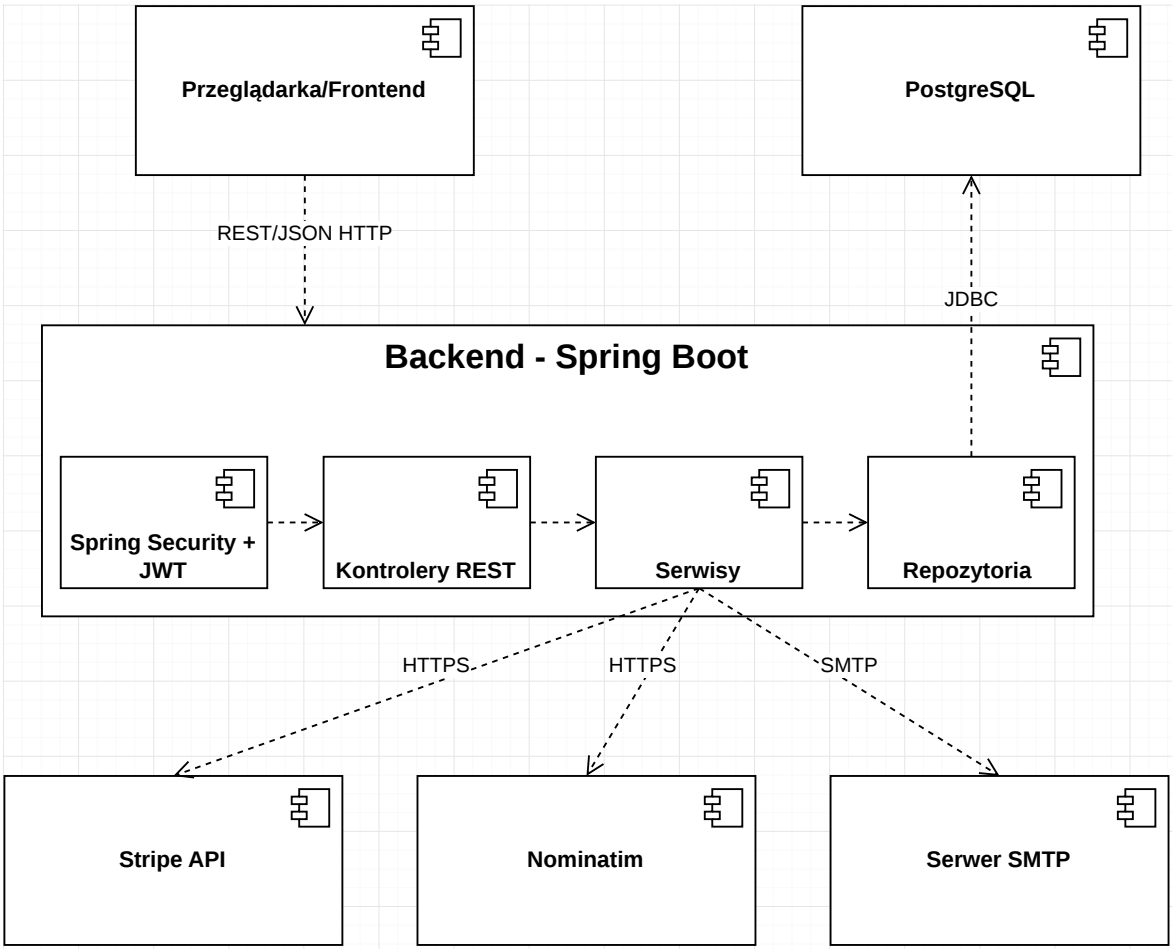
Specyfikacja:

1. Usługodawca podaje dane produktu: nazwę, opis, cenę, typy usług, adres oraz zdjęcia
2. System waliduje dane i zapisuje produkt
3. Usługodawca dodaje do produktu oferty: nazwę, opis, cenę, czas trwania i informację, czy możliwa jest płatność na miejscu
4. Usługodawca określa dostępność ofert: dni tygodnia oraz godziny rozpoczęcia i zakończenia
5. Produkt wraz z ofertami staje się widoczny dla klientów i można go rezerwować

Rozdział 4

Implementacja i dokumentacja techniczna

4.1. Diagram komponentów



Rysunek 4.1: Diagram komponentów systemu

4.1.1. Diagram komponentów

przedstawia z jakich warstw jest zbudowana aplikacja oraz cały przepływ danych pomiędzy tymi warstwami. Począwszy od akcji wykonywanych przez użytkownika w przeglądarce, kończąc na bazie danych, lub integracji z zewnętrznymi serwisami (np. Stripe, Nominatim, Serwer SMTP).

4.1.2. Przeglądarka/Frontend

Pierwszy etap przepływu danych dzieje się w przeglądarce internetowej. Interakcje z interfejsem UI, wykonywane przez użytkownika, są zbierane do formatu JSON, a następnie wysyłane poprzez sposób komunikacji z serwerem HTTP (aplikacja działa lokalnie).

4.1.3. Backend – Spring Security

Pierwszą warstwą, przez którą przepływają najpierw dane po otrzymaniu ich z klienta na serwer, jest Spring Security. Przed każdym wykonaniem się żądania, które wymaga potwierdzenia przez token autoryzujący JWT, backend sprawdza czy token jest poprawnie podpisany i nie wygaśnięty, jakie ma pozwolenia dostępu (permissions) do aplikacji oraz kto to jest. Token jest trzyczęściowym ciągiem znaków oddzielonych kropkami (header.payload.signature). **header** przechowuje zakodowane informacje o algorytmie używanym do hashowania części signature (w aplikacji jest to HS256), oraz typ tokenu, którym w tym przypadku jest JWT. **payload** zawiera tzw. claims, czyli dane o użytkowniku i samym tokenie. Owe claims zawierają dane takie jak: email użytkownika, jego rola uprawnień, czas wystawienia tokenu oraz czas wygaśnięcia (w aplikacji czas wygaśnięcia jest ustawiony na 30min). **signature** to zabezpieczenie integralności tokenu. Tworzy się go za pomocą algorytmu HMAC z funkcją skrótu SHA-256 (w tym przypadku), który jest algorytmem tzw. symetrycznym, czyli ten sam klucz jest używany do weryfikacji i podpisywania. Hashowanie przebiega używając pola payload (zakodowanego algorytmem Base64URL), header (również zakodowany w Base64URL), oraz secret[1]. Aby otrzymać signature, serwer łączy kropką zakodowane części header i payload, oblicza z nich kod HMAC-SHA256 z użyciem secretu, a wynik koduje algorytmem Base64URL. Przy każdym żądaniu wykorzystującym token, serwer na nowo liczy signature i porównuje ją.

[1] Jest to ciąg znaków, którego używa się, aby stworzyć signature, mający na celu zidentyfikowanie i autoryzację użytkowników wysyłających żądania do chronionych zasobów naszego serwera. Secret powinien być ściśle prywatny i chroniony przed dostaniem się w ręce nieupoważnionych osób, ponieważ znając secret, włamywacz mógłby samodzielnie wystawiać poprawnie podpisane tokeny i podszyć się pod dowolnego użytkownika w tym administratora

4.1.4. Backend - Kontrolery REST

Następnie po początkowym sprawdzeniu prawidłowości tokenu dołączonego do żądania wysłanego od klienta, dane są zarządzane przez warstwę tzw. kontrolerów. Jest to warstwa, która odpowiada za otrzymywanie żądań od klienta oraz późniejsze zwracanie wyniku. Następnie deleguje wykonanie potrzebnych działań do kolejnych warstw aplikacji (najczęściej jest to warstwa serwisów, która odpowiada przeważnie za implementację logiki biznesowej). W Spring Boot klasy kontrolerów oznacza się adnotacją @RestController. Każda metoda w kontrolerze odpowiada za konkretny typ żądania HTTP, które definiuje czy metoda będzie np. zapisywać dane do bazy danych (typ POST), wyszukiwać dane (GET), zmieniać dane (PUT/PATCH), lub usuwać (DELETE). Są to najczęściej używane oznaczenia do podstawowych operacji. Kolejną funkcjonalnością, która wchodzi w skład kontrolera jest walidacja danych wejściowych. Gdy dane od klienta trafiają do tej warstwy, jest sprawdzana poprawność wprowadzonych danych. Na przykład gdyby użytkownik wpisał nieprawidłowy format maila - zostałby wyrzucony wyjątek, jest to więc pierwsza linia obrony po stronie serwera w takich przypadkach. Warto wspomnieć również o tym jak metody zwracają dane do klienta. Używają do tego protokołu HTTP (tak samo jak klient gdy wysyła dane do backendu). W przypadku gdy metoda nie przewiduje zwracania żadnych danych, może zwrócić kod statusu, który informuje klienta o wyniku wykonanych działań w backendzie. Taki ACK najczęściej zawiera kod statusu. Przykładowe najczęściej spotykane statusy to: 200/OK (oznacza, że operacja przebiegła pomyślnie), 201/CREATED (informuje że zasób został zapisany pomyślnie), jak również statusy niepowodzenia takie jak: 400/BAD_REQUEST (często używany w np. w przypadku wpisania niepoprawnego formatu maila), 401/UNAUTHORIZED (gdy, użytkownik np. nie posiada poprawnego tokenu JWT), 403/FORBIDDEN (używany, gdy metoda jest chroniona i użytkownik nie ma do niej dostępu), 404/NOT_FOUND (w przypadku nie znalezienia zasobów), 409/CONFLICT (gdy np. termin rezerwacji jest już zajęty). Inaczej jest, gdy metoda ma za zadanie zwrócić klientowi dane. Robi to w formacie JSON, również używając protokołu HTTP.

Backend – Serwisy Warstwa serwisów jest kolejnym etapem przepływu danych. Odpowiadają one za logikę biznesową aplikacji. Są ściśle powiązane z kontrolerami, dzięki mechanizmowi tzw. wstrzykiwania zależności występującego w Spring Boot metody serwisowe mogą być bezpośrednio wywołane w warstwie kontrolera. Podczas wysłania żądania przez klienta do kontrolera, kontroler wywołuje metody serwisowe potrzebne do prawidłowego przeprowadzenia działań. W frameworku istnieje również bardzo ważna i często używana adnotacja **@Transactional**. Oznacza ona, że wszystkie operacje na bazie danych wykonane w ramach metody stanowią jedną transakcję, inaczej wszystkie operacje na bazie danych w danej metodzie muszą przejść pomyślnie. Jeśli jednak któraś operacja nie przebiegnie, wszystkie zmiany, które już nastąpiły się cofają i stan tabeli pozostaje taki jak przed wywołaniem metody. Jest to inaczej tzw. rollback. Takie podejście pozwala na zachowanie integralności danych. W przypadku tej aplikacji widać to dobrze na metodzie, która tworzy produkt. W środku metoda zapisuje najpierw kategorie produktu do tabeli z kategoriami, a dopiero potem tworzy się i zapisuje sam produkt, więc w przypadku pomyślnego zapisania kategorii, a niepowodzenia w zapisie produktu, aplikacja miałaby zbędne kategorie nieodpowiadające żadnemu produktowi. Na tym poziomie są wyrzucane również wyjątki, które są przechwytywane przez **GlobalExceptionHandler**. Jest to klasa która ma za zadanie znormalizować wyrzucane wyjątki, czyli zamiast dziesiątek linii kodu opisującego błąd, widoczny jest schludny komunikat zawierający takie informacje jak: klasa, w której wystąpił wyjątek, status HTTP wyjątku (np. 404), czas wystąpienia, URI żądania, podczas którego wystąpił wyjątek, i opcjonalnie komunikat ściślej opisujący wyjątek. Warto wspomnieć o niestandardowych wyjątkach występujących w aplikacji. Definiuje się je

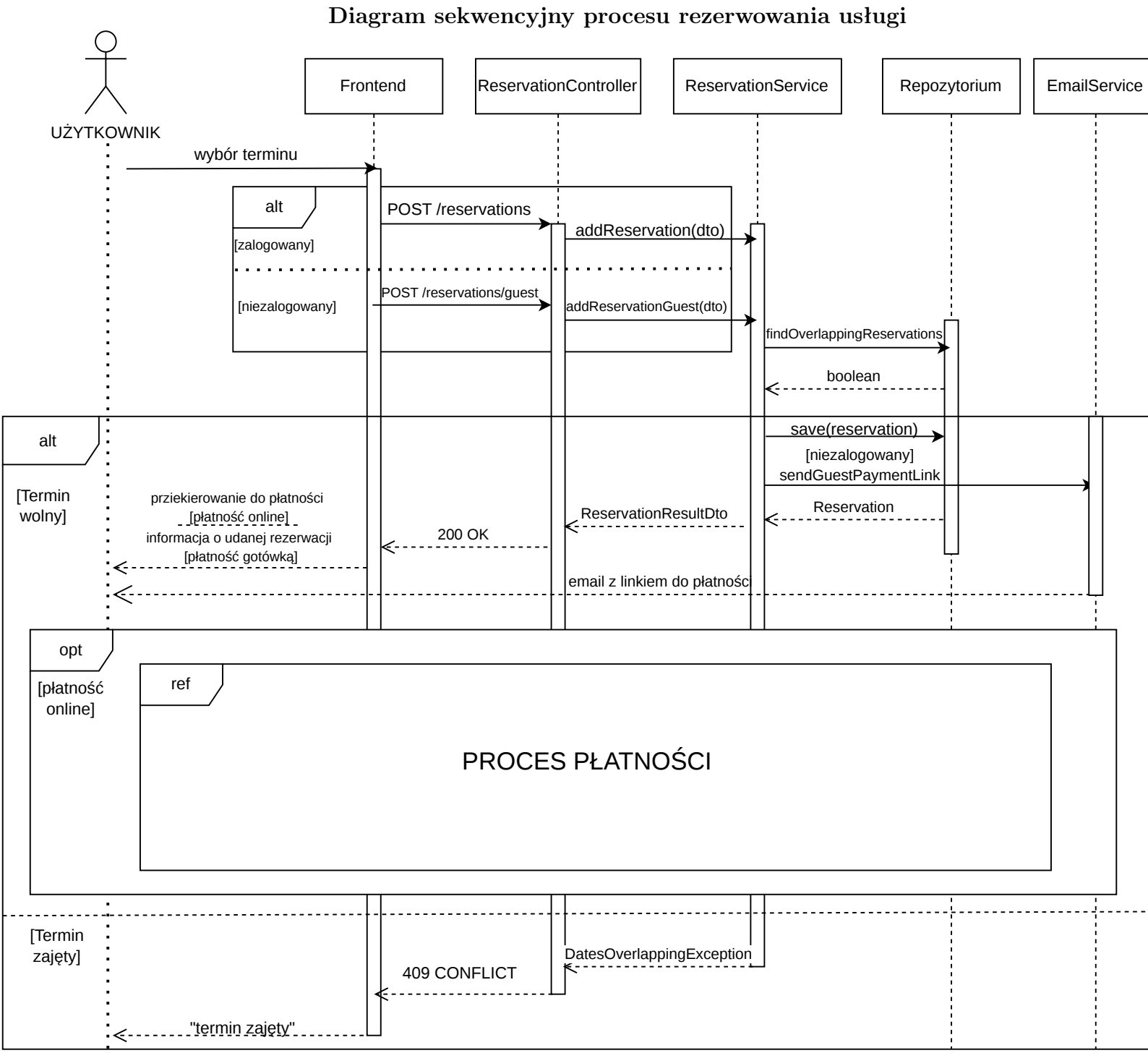
tworząc klasę, która dziedziczy po klasie **RuntimeException** (definiująca wyjątki wywołane podczas działania aplikacji). Następnie można ją również obsłużyć i znormalizować tak samo w **GlobalExceptionHandler**. Przykładem takiego niestandardowego wyjątku w aplikacji jest **DatesOverlappingException**. Wyjątek zostaje wyrzucony, gdy termin, na który użytkownik chce zarezerwować usługę, jest już zajęty lub nakłada się na istniejącą rezerwację. Kolejną rzeczą obsługiwaną w tej warstwie jest zmiana zwracanych danych klientowi, aby uniknąć wycieku wrażliwych danych lub po prostu aby metoda nie zwracała danych zbędnych dla klienta. Robi się to za pomocą DTO[2]. Definiuje się je w Javie w rekordach[3], zawierają one automatycznie equals(), hashCode() i toString(), akcesory o nazwie pola oraz konstruktor przyjmujący wszystkie pola, jak również tzw. kompaktowe konstruktory, dzięki którym można nadać walidację danych bez potrzeby przepisywania na nowo wszystkich pól. Pola w rekordzie są również niemutowalne, czyli finalne i prywatne. Rekordy poprawiają więc czytelność aplikacji i zmniejszają ilość zbędnego kodu. Aplikacja używa DTO praktycznie w każdej metodzie, która zwraca jakieś dane. Jest to szczególnie ważne w przypadku logowania lub rejestracji, aby nie dopuścić do sytuacji, w której metoda odpowiadająca za logowanie zwraca klientowi np. hasło. Mogłoby to poskutkować przejęciem hasła przez osobę nieuprawnioną, która mogłaby przechwycić ruch aplikacji. Nawet jeśli hasło jest już zahaszowane, byłoby to i tak niebezpieczne i niepożądane. Żeby temu zapobiec, w aplikacji jest używany DTO: **LoginResponse**, który zwraca: token JWT, nazwę użytkownika (email) i jego role. Serwisy są używane również do komunikacji z zewnętrznymi API. Aplikacja integruje się z trzema usługami zewnętrznymi: Stripe API (do obsługi płatności), Nominatim (To usługa geokodowania oparta na danych OpenStreetMap), Serwer SMTP (do połączenia z Google Gmail i wysyłania automatycznych maili).

[2] Obiekt transferu danych (DTO, ang. Data transfer object) – w programowaniu obiektowym, obiekt służący wyłącznie do przenoszenia danych między warstwami aplikacji, niezawierający logiki biznesowej.

[3] specjalny rodzaj klasy, który służy do tworzenia prostych, niemutowalnych obiektów będących nośnikami danych.

1. **Stripe API** jest to zewnętrzny serwis przeznaczony do obsługi płatności w aplikacji. Tworząc konto oraz używając trybu testowego w Stripe, użytkownik otrzymuje klucze. Kluczami istotnymi dla backendu są dwa: jednym z nich jest klucz prywatny (secret_key), który daje dostęp do komunikacji oraz utworzenia sesji płatności w Stripe. Drugim zaś jest prywatny klucz webhook, który pozwala serwerowi Stripe na wysłanie informacji do backendu aplikacji, gdy np. opłata przebiegła pomyślnie lub zakończyła się niepowodzeniem. Następnie ta informacja trafia do dedykowanej metody w aplikacji, która ma za zadanie ustawić status rezerwacji na potwierdzony, lub odrzucony w bazie danych. Używana jest do tego specjalna biblioteka stworzona do Java – stripe-java.
2. **Nominatim** w aplikacji jest używany do geokodowania. Tworząc produkt, użytkownik ma możliwość wpisania adresu, w którym będzie odbywała się usługa, lub siedziby podmiotu wykonującego usługę. Następnie komunikując się z Nominatim API, gdy adres jest poprawny i został odnaleziony, zostaje wykonane geokodowanie tego adresu. Następnie jest przekształcany na współrzędne geograficzne, które są potem zapisywane wraz ze stworzonym produktem. Użytkownik ma również możliwość filtrowania usług według miast. Aplikacja komunikuje się z Nominatim API, tworząc URI zgodny z formatem żądań Nominatim z parametrem miasta wpisanego przez użytkownika. Następnie zwracane są wszystkie miasta znalezione o tej nazwie. Warto wspomnieć, że w przypadku, gdy znaleziono więcej niż 10 miast, API zwróci maksymalnie 10, wynika to z celowego ustawienia limitu podczas tworzenia URI po stronie backendu.
3. **Serwer SMTP** protokół działający w ramach biblioteki spring-boot-starter-mail, stworzona do wysyłania automatycznych wiadomości email z powiązanego konta np. Gmail. W aplikacji, maile są wysyłane do użytkowników w trzech przypadkach: potwierdzenie opłacenia usługi, przypomnienie o wizycie 24 godziny przed jej terminem oraz wiadomość z linkiem do kontynuowania płatności dla użytkowników niezalogowanych.

4.2. Diagramy sekwencyjne



Rysunek 4.2: Diagram sekwencyjny procesu rezerwacji

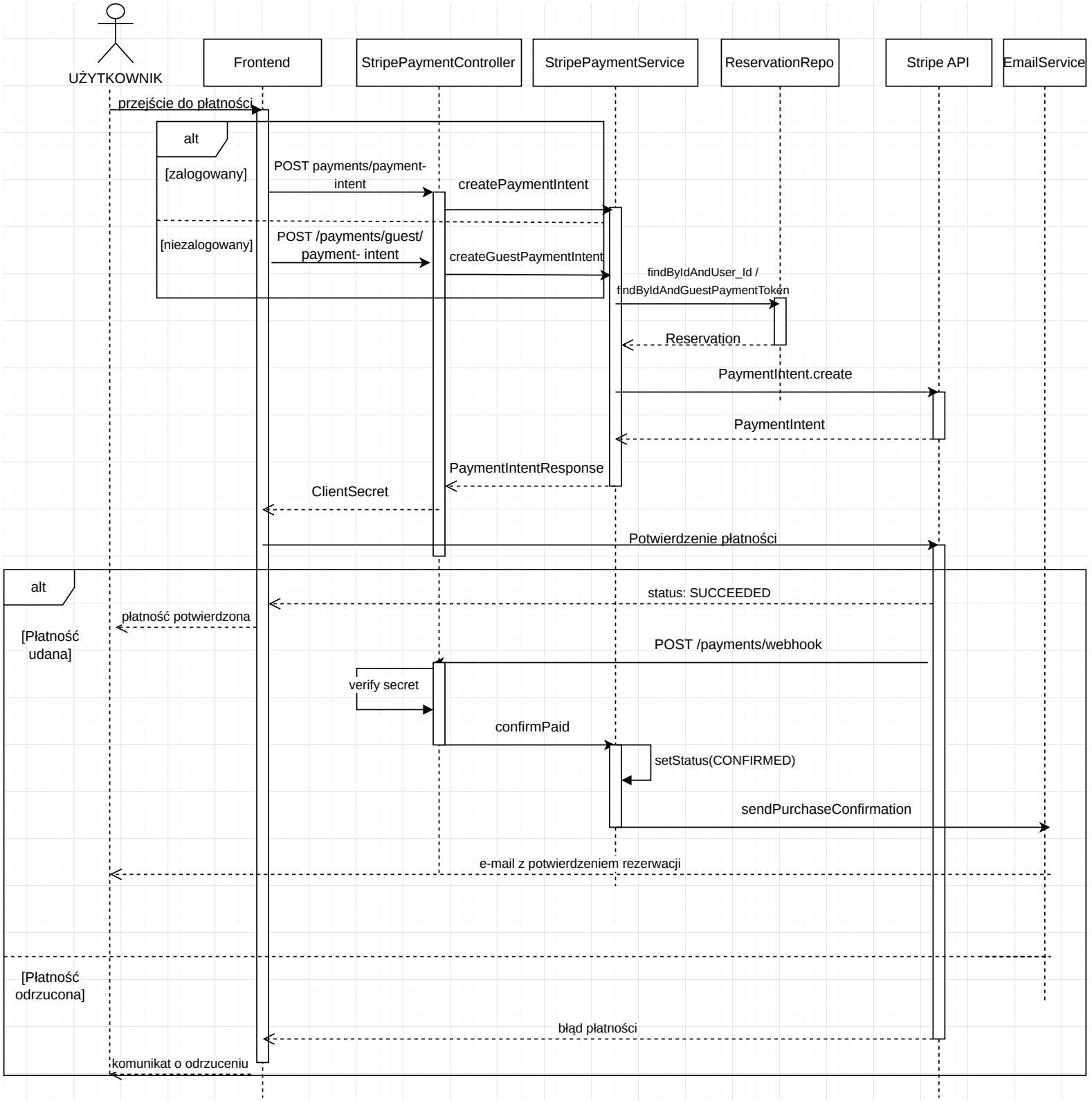
Jednym z kluczowych procesów zachodzących w aplikacji jest rezerwowanie. Na powyższym diagramie został on szczegółowo przedstawiony. Po wyborze oferty użytkownikowi jest przedstawiany kalendarz, w którym użytkownik może wybrać dzień z dostępnymi terminami. Po kliknięciu w dany dzień jest następnie wyświetlana lista dostępnych terminów na ten dzień (jest ona tworzona na podstawie pola „duration_minutes" w tabeli PRODUCT_OFFERS, która definiuje szacowaną długość wykonywania usługi, oraz pól „day_of_week", „start_time" i „end_time" w tabeli PRODUCT_OFFER_AVAILABILITIES, połączonej z PRODUCT_OFFERS relacją wiele do jednego. Te pola ustalają ramy czasowe, w których może być wykonana rezerwacja. Na koniec jest wyświetlana lista przycisków z godzinami dostępnych terminów obliczonymi na podstawie tych pól). Przy każdym terminie istnieje również opcja prośby o fakturę (gdy usługodawca pozwala na taką możliwość)[4]. Po kliknięciu w wybrany termin i zaakceptowaniu rezerwacji użytkownikowi zalogowanemu jest przedstawiane okno wyboru płatności (gotówkowo lub online), zaś użytkownicy niezalogowani (goście) zostają przekierowani na stronę formularza wypełnienia danych osobowych. Kolejnym krokiem jest wysłanie żądania przez klienta do serwera na endpointy odpowiedzialne za tworzenie rezerwacji w warstwie kontrolerów. Logika tworzenia rezerwacji jest rozdzielona, więc różna dla użytkowników zalogowanych i niezalogowanych (gości). Z tego względu istnieją dwa endpointy POST/reservations oraz POST/reservations/guest. Powodem takiego podziału jest głównie różnica wymaganych danych wprowadzanych do formularza przez użytkowników. W przypadku gościa musi on podać swoje imię, nazwisko oraz e-mail, zaś gdy użytkownik jest zalogowany, jego dane są pozyskiwane dzięki tokenowi JWT. Co więcej, możliwość płatności gotówkowej jest dostępna tylko dla użytkowników zalogowanych, goście nie mają takiej możliwości, aby ograniczyć masowe rezerwacje danej usługi przez osoby, które nie mają na celu skorzystania z usługi, a chcą np. przeciążyć system lub specjalnie zablokować dany termin innym usługobiorcom. Takie podejście ma sens, ponieważ nieopłacone rezerwacje bezgotówkowe są usuwane z systemu automatycznie po 15 minutach, zapewnia to pole w rezerwacji „payment_status", które zmienia się z „PENDING_PAYMENT" na „CONFIRMED" po uiszczeniu opłaty lub „CANCELLED" przy odrzuceniu. Metoda odpowiadająca za usuwanie nieopłaconych rezerwacji po 15 minutach, naturalnie bierze pod uwagę również to, czy użytkownik chce zapłacić gotówką, czy online, i usuwa pod warunkiem, że pole „PAYMENT_METHOD" w rezerwacji jest „CARD". Po otrzymaniu żądania są sprawdzane podstawowe zabezpieczenia (np. w przypadku gości – czy dane osobowe mieszczą się w ustalonych wcześniej ramach), jeśli tak – operacja przechodzi do kolejnego etapu, jakim jest obsługa logiki biznesowej. Wywoływana jest metoda addReservation lub addReservationGuest (dla gości), która w parametrze otrzymuje dane użytkownika oraz opcjonalnie faktury. Komunikując się z bazą danych, warstwa serwisów wywołuje przeszukanie bazy danych za pomocą metody findOverlappingReservations, która zwraca boolean, sprawdzając, czy rezerwacja w tym terminie jest już zajęta, i jeśli tak, to na poziomie serwisu jest zgłaszany wyjątek „DatesOverlappingException", który potem jest mapowany w kontrolerze na błąd o statusie 409 (CONFLICT); komunikat zwracany użytkownikowi w tym przypadku brzmi: „termin zajęty". Taki mechanizm jest bardzo ważny projektowo, ponieważ po pierwsze chroni przed sytuacją, w której użytkownik samodzielnie wyśle zapytanie do aplikacji, nie używając interfejsu, unikając przy tym zabezpieczeń po stronie frontendu. Po drugie, gdy dwóch użytkowników spróbuje zarezerwować usługę w tym samym momencie na ten sam termin, mogłoby to poskutkować utworzeniem się dwóch rezerwacji na ten sam termin. Natomiast to nie rozwiązuje całkowicie tego problemu, ponieważ używana w aplikacji baza danych, jaką jest PostgreSQL, ma do-

[4] Ze względu na czytelność diagramu pominięto logikę tworzenia i zapisu danych faktury.

myślny poziom izolacji[5] „READ_COMMITTED", więc metoda findOverlappingReservations będzie widziała dane w bazie tylko te, które zostały już zatwierdzone, czyli gdzie transakcja przebiegła pomyślnie. W takim przypadku, gdyby dwie osoby zarezerwowały ten sam termin, mogłoby to również poskutkować stworzeniem się dwóch rezerwacji. W aplikacji przy metodach zapisu rezerwacji jest ustawiony więc inny poziom izolacji, jakim jest „SERIALIZABLE". Przy takim podejściu transakcja jest wycofywana, gdy dane, które odczytała, zostały w międzyczasie zmienione przez inną transakcję. Przy faktycznym wycofaniu się transakcji z tego powodu, zostanie zgłoszony wyjątek ConcurrencyFailureException, który w aplikacji został obsłużony i zwracany użytkownikowi jako komunikat informujący o zajęciu terminu przez kogoś innego. Natomiast gdy termin jest wolny, rezerwacja się zapisuje, zwracając specjalnie przygotowane DTO z danymi rezerwacji, a użytkownik po stronie interfejsu zostaje przekierowany do płatności (z wyjątkiem zalogowanych użytkowników, którzy zdecydowali się na płatność gotówką; w takim przypadku zostaje wyświetlona informacja o pomyślnym złożeniu rezerwacji). Ponadto po wykonaniu zapisu rezerwacji jest również wysyłany mail do niezalogowanych użytkowników, pozwalający użytkownikom kontynuować płatność. Ma to na celu zapobiegać sytuacji, w której okno płatności zostanie zamknięte przed opłaceniem usługi. W mailu zawarty jest link tworzący nową transakcję oraz przekierowujący użytkownika do formularza płatniczego. Każda rezerwacja gościa posiada token `GuestPaymentToken` (UUID), który stanowi część linku do dokończenia płatności. Token jednoznacznie identyfikuje rezerwację bez konieczności logowania i jest trudny do odgadnięcia przez osoby trzecie. Dodatkowo przed utworzeniem transakcji sprawdzany jest status rezerwacji, dzięki czemu link nie działa dla rezerwacji już opłaconych, co zapobiega podwójnej płatności. W przypadku użytkowników zalogowanych taki mail nie zostaje wysłany, ponieważ użytkownicy mogą przejść do dokończenia płatności w panelu użytkownika, w którym są wylistowane ich rezerwacje z opcją dokonania opłacenia. Trzeba zaznaczyć, że metoda sendGuestPaymentLink, widniejąca na diagramie, jest wywoływana tylko w przypadku niezalogowanych użytkowników, z powodu opisanego wcześniej. Proces płatności online został opisany na kolejnym diagramie poniżej.

[5] Poziom izolacji transakcji określa, jakie zmiany innych równoległych transakcji na bazie danych są widoczne dla danej transakcji oraz jak baza reaguje na konflikty między nimi.

Diagram sekwencyjny procesu płatności

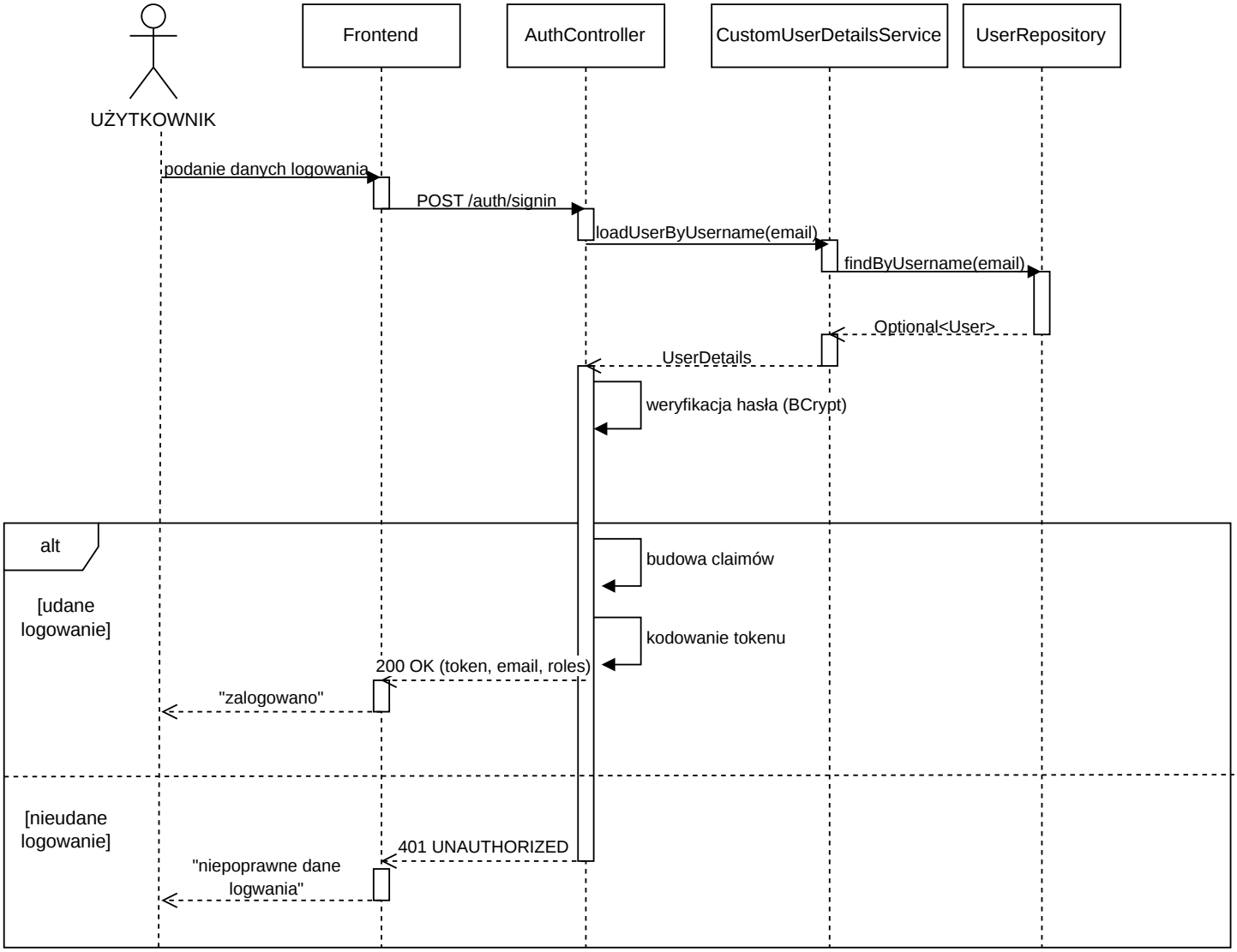


Rysunek 4.3: Diagram sekwencyjny procesu płatności

Kolejnym procesem, który jest rdzeniem aplikacji, jest proces płatności. Wszyscy użytkownicy mają możliwość zapłacenia kartą, a zalogowani użytkownicy mogą również wybrać płatność gotówkową. Płatnościami, ich autoryzacją i przetwarzaniem zajmuje się zewnętrzny serwis – Stripe. Komunikując się z nim, aplikacja jest w stanie operować danymi wewnątrz bazy danych. Diagram przedstawiony powyżej przedstawia proces wykonujący się po stworzeniu rezerwacji. Po pomyślnym zarezerwowaniu terminu następuje komunikacja ze stroną backendu. W zależności od tego, czy użytkownik jest zalogowany, czy nie, są wysyłane żądania do danego endpointu. Dla zalogowanych użytkowników żądanie trafia do POST payments/payment-intent, a w przypadku niezalogowanych do POST payments/guest/payment-intent. Odpowiedzialne są za obsługę obiektów płatności w Stripe. Logika endpointów jest rozdzielona celowo: w przypadku gości (niezalogowanych użytkowników) podczas tworzenia rezerwacji generowany jest również token identyfikacji płatności przy każdej rezerwacji gościa. Bez takiego tokenu ktoś mógłby odgadnąć id rezerwacji, przez co mógłby wykonać pewne operacje na tej rezerwacji. Z długim tokenem UUID trzeba odgadnąć id oraz token, co jest niezwykle ciężkie do odgadnięcia. W przypadku zalogowanych użytkowników takiego problemu nie ma, ponieważ rezerwację szukamy za pomocą tokenu JWT oraz id rezerwacji. Aby zrozumieć całkowite działanie procesu płatności, trzeba cofnąć się do etapu tworzenia rezerwacji. Rezerwacja jest tworzona przy użyciu osobnego endpointu i metod przed rozpoczęciem procesu płatności. Po stworzeniu rezerwacji obiekt zarezerwowanej usługi jest zwracany do klienta, który, otrzymując te dane, rozpoczyna płatność, wysyłając żądanie do backendu na wspomniane wcześniej endpointy tworzące payment-intent, czyli obiekt tworzony na serwerze Stripe, noszący dane o płatności. Metody najpierw wyszukują odpowiadającą płatności rezerwację, ponieważ to ona przechowuje informacje o np. cenie usługi, która jest niezbędna do utworzenia payment-intent. W przypadku nieznalezienia odpowiadającej rezerwacji wyrzucany jest wyjątek. W rezerwacji istnieje również pole „stripePaymentIntentId", metoda więc sprawdza, czy to pole nie jest null. Gdyby wynikło, że faktycznie nie jest null, wyszukiwany byłby obiekt intent na serwerze Stripe, na podstawie wcześniej wspomnianego pola „stripePaymentIntentId". Biblioteka „stripe-java" ma wiele wbudowanych metod. Jedną z nich jest „PaymentIntent.retrieve", właśnie ona jest odpowiedzialna za wyszukiwanie payment-intent, komunikując się ze Stripe API. Podczas wykonywania się tego typu metod z biblioteki „stripe-java" program wie dokładnie, z którym środowiskiem testowym Stripe ma się skomunikować, ponieważ tworząc środowisko w Stripe, twórca aplikacji otrzymuje tzw. „STRIPE_SECRET_KEY", który jest unikalnym tajnym kluczem API pozwalającym właśnie na komunikację. W aplikacji tworzony jest Bean konfiguracyjny, który zawiera właśnie to ID. Gdy metoda „PaymentIntent.retrieve" faktycznie znajdzie już obiekt Stripe o tym „stripePaymentIntentId", zwracany do klienta zostaje ID intentu oraz „ClientSecret". ClientSecret jest to klucz tworzony przez Stripe przy każdym tworzeniu nowego obiektu intent, jest on bardzo ważny, ponieważ pozwala on na weryfikację, czy płatność pochodzi z prawdziwego serwera Stripe, a nie jest podrabiana. Następnie użytkownik zostaje automatycznie przekierowany do okna płatności. Po stronie klienta używana jest biblioteka do Reacta od Stripe – „stripe-js", która pozwala na komunikację ze Stripe API i obsługę płatności kartą. Następnie informacje o karcie płatniczej są dostarczane do testowego środowiska Stripe, które waliduje ważność karty, weryfikuje ją oraz księguje, zmieniając odpowiednio status płatności, np. na „succeeded", czyli sukces. Weryfikacja jest możliwa właśnie dzięki ClientSecret, który wraz z danymi karty oraz stripePaymentIntentId jest wysyłany do Stripe, następnie tam jest weryfikowane, czy sekret wysłany od klienta pokrywa się z tym podpiętym do odpowiadającego obiektu intent. Warto zaznaczyć, że dane karty nie mają kontaktu ze stroną backendu, czyli frontend od razu wysyła dane do środowiska Stripe. Jest to ważne z punktu widzenia bezpieczeństwa danych: aplikacja oddaje całość przetwarzania płatności do Stripe, aby uniknąć np. wycieku danych kart po stro-

nie backendu, oraz dzięki temu aplikacja ogranicza zakres wymagań PCI DSS. Bezpieczeństwo danych aplikacja przekazuje do zewnętrznego serwisu z dobrą reputacją i dużym skupieniem na zabezpieczeniach przed włamaniami. Z powodu tego, że aplikacja działa w środowisku testowym, prawdziwe dane kart nie działają. W razie potrzeby sprawdzenia, czy płatności w aplikacji działają poprawnie, powinno się użyć domyślnych danych karty, które będą zaakceptowane przez Stripe w tym przypadku, czyli: 4242 4242 4242 4242 oraz dowolny numer CVV. Przy pomyślnym potwierdzeniu płatności użytkownik dostaje informację zwrotną o pomyślnej płatności, a do backendu zostaje wysłany automatycznie webhook, który aplikacja nasłuchuje. Webhook wysyła żądanie na endpoint POST payments/webhook. Aplikacja weryfikuje, czy webhook pochodzi od Stripe i nie został zmodyfikowany, ponieważ we właściwościach aplikacji jest zdefiniowany „STRIPE_WEBHOOK_SECRET", czyli klucz, który jest generowany dla każdego środowiska Stripe i na jego podstawie jest ustalana wiarygodność otrzymywanego webhooka. Żądanie od Stripe zawiera między innymi reservationId, co pozwoli zidentyfikować rezerwację i zaktualizować jej status. Następnie wywołana zostaje metoda „confirmPaid", która na podstawie wartości metadanych otrzymanych przez serwer od Stripe oznaczy rezerwację jako taką, której płatność się powiodła, lub nie zmieni statusu, gdy płatność się nie powiodła, zwracając przy tym odpowiednie komunikaty sukcesu lub błędu. Przy pomyślnym przebiegu jest wysyłany mail z potwierdzeniem.

Diagram sekwencyjny procesu logowania



Ostatnim diagramem sekwencyjnym przedstawionym w pracy jest proces logowania. Budowa tokenu JWT oraz mechanizm jego weryfikacji zostały opisane w podrozdziale 4.1.3. Niezalogowany użytkownik nadal może korzystać z wielu funkcji aplikacji takich jak: Wyszukiwanie rezerwacji i ich filtrowanie, rezerwowanie oraz płatność online. Jednakże posiadanie konta w aplikacji przynosi wiele użyteczności. W aplikacji w pasku nawigacji na górze strony widnieją dwie zakładki: „signin"(zaloguj się) i „signup"(zarejestruj się). Okno rejestracji oraz logowania wygląda praktycznie tak samo, oraz naturalnie, żeby móc się zalogować, najpierw trzeba się zarejestrować. Po wpisaniu w zakładce rejestracji prawidłowego maila i hasła, które w aplikacji musi mieć co najmniej 8 znaków, konto użytkownika jest założone. Użytkownik nie jest jednak jeszcze zalogowany. Aby się zalogować, trzeba przejść do zakładki „signin" i wypełnić pola „email" i „hasło" danymi wpisanymi przy rejestracji. Po wpisaniu danych i kliknięciu przycisku "signin", zostaje wysłane żądanie do serwera na endpoint POST auth/signin. Na tym etapie użytkownik jest uwierzytelniany. Metoda „authenticate" z biblioteki spring-security, działa wewnątrz tzw. AuthenticationManager, czyli wewnętrzny mechanizm, który zajmuje się obsługą uwierzytelnianie żądań zalogowania. AuthenticationManager przekazuje dane użytkownika do DaoAuthenticationProvider, który następnie wywołuje metodę „loadUserByUsername", która jest używana do wyszukania użytkownika w bazie. Jest ona wbudowaną metodą spring security. W aplikacji jest implementowana używając adnotacji @Override, aby zdefiniować na jakiej podstawie jest wyszukiwany użytkownik. W tym przypadku jest to pole „username", które zawiera email użytkownika. Metoda „authenticate" odpowiada również za weryfikację hasła: hasło wpisane przez użytkownika jest porównywane z hashem BCrypt zapisanym w bazie (salt jest przechowywany wewnątrz hasha). Jeśli hasło jest poprawne, proces przechodzi do etapu budowy tzw. claims, czyli danych o użytkowniku, które token JWT będzie przenosił, a następnie do zakodowania i podpisania tokenu. Po prawidłowej weryfikacji do klienta jest zwracany komunikat o pomyślnym zalogowaniu wraz z tokenem JWT, emailem i rolami. W przypadku, gdyby, użytkownik w bazie nie został znaleziony, lub hasło byłoby niepoprawne, zostałby wyłapany wyjątek o statusie 401 oraz komunikat.

Rozdział 5

Testy i wyniki

5.1. Testowanie

W aplikacji zastosowano testy jednostkowe z użyciem atrap (mocków)[1] oraz testy integracyjne[2], wykonywane na prawdziwej bazie PostgreSQL uruchamianej w kontenerze Docker za pomocą biblioteki Testcontainers[3]. Pokrycie testów jest generowane w raporcie dostępnym również w formacie HTML przy użyciu biblioteki JaCoCo. Całkowite pokrycie instrukcji wynosi 44%. W pliku `pom.xml` wykluczono z raportu pliki, takie jak DTO lub mappery (klasy zajmujące się mapowaniem obiektów na DTO), ponieważ nie zawierają logiki biznesowej i nie powinny wpływać na całkowite pokrycie funkcjonalności aplikacji. Skupiono się na najważniejszych aspektach aplikacji, takich jak rezerwacje, płatności, autentykacja oraz fakturowanie. Warto zaznaczyć, że do testów jest stworzony oddzielny plik application.properties, w którym definiujemy testowe dane potrzebne np. do sprawdzenia czy wysyłanie maili działa lub webhook stripe zwraca prawidłowe dane.

[1] Mock objects (mocki) w testach to obiekty naśladujące zachowanie prawdziwych obiektów w systemie. Służą do symulowania zachowania zależności lub współpracujących komponentów kodu.

[2] Testy integracyjne sprawdzają, czy moduły, usługi i systemy poprawnie ze sobą rozmawiają: czy dane przekazywane przez interfejs mają właściwy format, znaczenie i kolejność.

[3] Testcontainers to biblioteka umożliwiająca uruchamianie w trakcie testów jednorazowych kontenerów Docker (np. baz danych). Dzięki temu testy wykonują się na tym samym silniku bazy, który działa w aplikacji, a środowisko testowe jest izolowane i powtarzalne.

5.1.1. Testy jednostkowe

W aplikacji jest zawarta paczka bibliotek „spring-boot-starter-test", zawiera ona w sobie potrzebne podzespoły do przeprowadzenia testów jednostkowych z użyciem mocków. Pierwszym z nich jest JUnit 5, jest to framework, który między innymi inicjalizuje klasy testów jednostkowych, dostarcza wielu potrzebnych narzędzi do testowania, takich jak np. adnotacja @BeforeEach, która pozwala na wykonanie pewnych czynności przed uruchomieniem się każdego testu (np. stworzenie użytkownika/rezerwacji). JUnit daje również możliwość wykonywania asercji, czyli porównań wyników, np. aby sprawdzić, czy pożądany wynik jest rzeczywiście prawdziwy lub czy wyrzucony wyjątek jest taki jak oczekiwany. Jednakże w aplikacji użyto innego pakietu do asercji: assertj.core.api.Assertions.assertThat, ponieważ wyświetla czytelniejsze komunikaty błędów. W paczce jest również zawarta biblioteka „Mockito". Biblioteka daje adnotacje @Mock oraz @InjectMocks, które ułatwiają i przyspieszają użycie mocków w testach, ponieważ nie trzeba samemu ich tworzyć i za każdym razem wstrzykiwać. Dostarcza również metodę `when` oraz metody `thenReturn` i `thenThrow`, które pozwalają określić, że przy danym działaniu funkcja powinna zwrócić dany wynik albo wykonać dane konkretne działanie. Obie części chodzą w parze: `when` określa, kiedy coś się stanie, a `thenReturn` (zwrócenie wyniku) lub `thenThrow` (zgłoszenie wyjątku) określa, że ma to być wynikowe działanie dla `when`. W aplikacji testami jednostkowymi objęto przede wszystkim warstwę serwisów: wykrywanie kolizji terminów rezerwacji, reguły dotyczące metod płatności dla gości i zalogowanych użytkowników, zmiany statusów rezerwacji oraz zgłaszanie wyjątków w przypadkach błędnych danych. Wszystkie przypadki testowe zestawiono w tabeli 5.1.

5.1.2. Testy integracyjne

W aplikacji szczególnie skupiono się na testach integracyjnych ze względu na charakter systemu, który łączy się z wieloma API oraz komunikuje się w metodach z wieloma komponentami serwisowymi na raz (np. rezerwacje z ofertami oraz fakturami). Paczka bibliotek „spring-boot-starter-test" dostarcza również wiele bibliotek do przeprowadzenia testów integracyjnych. Jednakże jest też potrzebna biblioteka do testowania użytkowników wraz z tokenem JWT, autoryzacją i poziomami dostępu. Do tego użyto biblioteki „spring-security-test". Kolejną potrzebną w tym etapie biblioteką jest `spring-boot-testcontainers`, która pozwala uruchomić bazę danych PostgreSQL w kontenerze Docker i automatycznie podłączyć ją do aplikacji na czas testów. Kontenerem jest wyłącznie baza danych; samą aplikację uruchamia Spring w ramach `@SpringBootTest`. Do wykonania testów integracyjnych wymagany jest działający Docker, natomiast nie trzeba ręcznie uruchamiać bazy ani aplikacji. Przy rozpoczęciu testów integracyjnych inicjalizowana jest klasa „AbstractIntegrationTest", która używając `PostgreSQLContainer` tworzy testowy serwer bazy danych PostgreSQL. Następnie używana jest komenda `TRUNCATE TABLE users, products, product_offers, reservations, invoice_requests RESTART IDENTITY CASCADE`, która usuwa wszystkie dane z wymienionych tabel oraz tabel powiązanych z nimi kluczami obcymi i resetuje sekwencje identyfikatorów, aby stare dane nie powodowały sprzeczności z już istniejącymi. Ta komenda wykonuje się przed każdym testem. W tej klasie istnieją również metody oznaczone jako „protected", czyli mające dostępność w obrębie tego samego pakietu lub w klasach dziedziczących. Klasy testowe dziedziczą po „AbstractIntegrationTest", więc będą miały do tych metod dostęp. Jedna z nich tworzy użytkownika, a druga ofertę. Mają one na celu poprawienie czytelności oraz usprawnienie prędkości pisania testów, ponieważ będą użyte w wielu różnych klasach testowych odpowiadających za testowanie różnych aspektów aplikacji (np. autoryzacji i rezerwacji). Testy integracyjne obejmują trzy różne klasy. Pierwsza testuje przepływ danych podczas autentykacji oraz logowania użytkownika, druga rezerwacje, a trzecia poprawność przepływu webhooków i całej obsługi płatności. Ponownie testy zostały opisane w tabeli 5.2.

Tabela 5.1: Testy jednostkowe

ID	Krok	Oczekiwany wynik	Wynik
ReservationServiceTest			
J1	Zalogowany użytkownik rezerwuje wolny termin	Zapis rezerwacji ze statusem `PENDING_PAYMENT`, brak użycia repozytorium faktur	Zaliczony
J2	Rezerwacja z danymi faktury dla oferty z fakturami	Zapisane żądanie faktury powiązane z rezerwacją	Zaliczony
J3	Rezerwacja z fakturą dla oferty bez faktur	`InvoiceUnavailableException`, brak zapisu rezerwacji	Zaliczony
J4	Rezerwacja z terminem w przeszłości	`DateInThePastException`, brak wywołań repozytoriów	Zaliczony
J5	Termin nakłada się na istniejącą rezerwację	`DatesOverlappingException`, brak zapisu	Zaliczony
J6	Rezerwacja nieistniejącej oferty	`NoSuchElementException`	Zaliczony
J7	Rezerwacja gościa na wolny termin	Rezerwacja bez użytkownika, zapisany e-mail gościa, token UUID (36 znaków), wysłany link do płatności	Zaliczony
J8	Rezerwacja gościa na zajęty termin	`DatesOverlappingException`, e-mail nie zostaje wysłany	Zaliczony
J9	Właściciel produktu odwołuje rezerwację	Rezerwacja zostaje usunięta	Zaliczony
J10	Użytkownik niebędący właścicielem odwołuje rezerwację	`OwnershipViolationException`, rezerwacja nie jest usuwana	Zaliczony
J11	Usunięcie rezerwacji innego użytkownika	`NoSuchElementException`, brak usunięcia	Zaliczony
J12	Uruchomienie czyszczenia wygasłych rezerwacji	Repozytorium wywołane ze statusem `CONFIRMED` i bieżącym czasem	Zaliczony
InvoiceRequestServiceTest			
J13	Utworzenie faktury dla oferty bez faktur	`InvoiceUnavailableException`, brak zapisu	Zaliczony
J14	Właściciel oznacza zażądaną fakturę jako wystawioną	Status `ISSUED`, ustawiona data wystawienia	Zaliczony
J15	Właściciel ponownie oznacza wystawioną fakturę	Powrót do statusu `REQUESTED`, data wystawienia wyczyszczona	Zaliczony
J16	Zmiana statusu faktury przez nie-właściciela	`OwnershipViolationException`, status bez zmian	Zaliczony
StripePaymentServiceTest			
J17	Utworzenie płatności dla rezerwacji o cenie 79,99	Kwota 7999 groszy, waluta `pln`, metadane z `reservationId`, identyfikator zapisany w rezerwacji	Zaliczony
J18	Utworzenie płatności, gdy istnieje aktywny intent	Intent zostaje użyty ponownie, nowy nie jest tworzony	Zaliczony
J19	Utworzenie płatności, gdy poprzedni intent jest anulowany	Utworzony nowy intent, jego identyfikator zapisany w rezerwacji	Zaliczony
J20	Płatność za rezerwację już opłaconą	`IllegalStateException`	Zaliczony
J21	Płatność za rezerwację innego użytkownika	`NoSuchElementException`	Zaliczony
J22	Płatność gościa z błędnym tokenem	`NoSuchElementException`	Zaliczony
J23	Płatność gościa po upływie okna płatności	`IllegalStateException`, brak komunikacji ze Stripe	Zaliczony
J24	Płatność gościa z poprawnym tokenem	Utworzony intent, identyfikator zapisany w rezerwacji	Zaliczony
J25	Potwierdzenie opłaconej rezerwacji	Status `CONFIRMED`, wyczyszczony termin płatności, wysłany e-mail z potwierdzeniem	Zaliczony
J26	Potwierdzenie płatności dla rezerwacji anulowanej	Status bez zmian, e-mail nie zostaje wysłany	Zaliczony

Tabela 5.2: Testy integracyjne

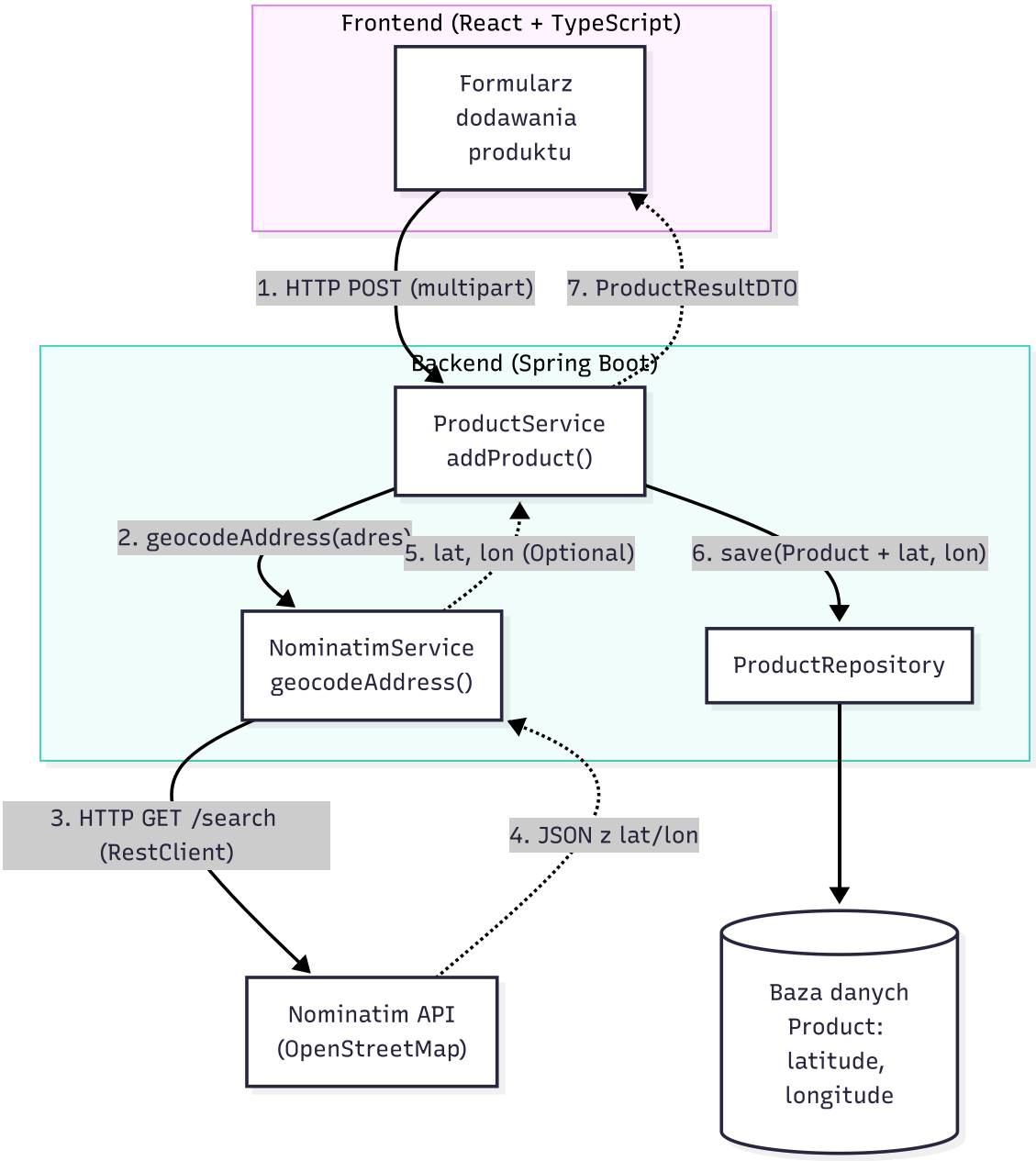
ID	Krok	Oczekiwany wynik	Wynik
AuthFlowIntegrationTest			
I1	Rejestracja, logowanie i wywołanie chronionego endpointu wydanym tokenem	Hasło zapisane jako skrót, logowanie zwraca e-mail, rolę `USER` i token, endpoint z tokenem zwraca status 200	Zaliczony
I2	Rejestracja na zajęty adres e-mail	Status 400, w bazie pozostaje jeden użytkownik	Zaliczony
I3	Logowanie z błędnym hasłem	Status 401, komunikat `Bad credentials`	Zaliczony
I4	Chroniony endpoint bez tokenu i z fałszywym tokenem	Status 401 w obu przypadkach	Zaliczony
ReservationFlowIntegrationTest			
I5	Użytkownik tworzy rezerwację	Status 200, rezerwacja zapisana w bazie ze statusem `PENDING_PAYMENT` i terminem płatności za około 15 minut	Zaliczony
I6	Rezerwacja z danymi faktury	Status 200, żądanie faktury zapisane w bazie	Zaliczony
I7	Rezerwacja nakładająca się na istniejącą oraz rezerwacja bezpośrednio po niej	Nakładająca się: status 409, następna: status 200, w bazie dwie rezerwacje	Zaliczony
I8	Rezerwacja z terminem w przeszłości	Status 403, brak rezerwacji w bazie	Zaliczony
I9	Rezerwacja bez tokenu	Status 401	Zaliczony
I10	Rezerwacja gościa bez logowania	Status 200, rezerwacja z tokenem płatności w bazie, e-mail z linkiem zawierającym token	Zaliczony
I11	Odwołanie rezerwacji przez obcego usługodawcę, a potem przez właściciela	Obcy: status 409 i rezerwacja pozostaje, właściciel: status 200 i rezerwacja usunięta	Zaliczony
I12	Czyszczenie wygasłych rezerwacji	Usunięta tylko wygasła rezerwacja kartą, pozostałe (gotówka, potwierdzona, jeszcze płatna) zachowane, brak e-maili	Zaliczony
StripeWebhookIntegrationTest			
I13	Webhook `payment_intent.succeeded` z poprawnym podpisem	Status 200, rezerwacja `CONFIRMED`, wyczyszczony termin płatności, e-mail do klienta	Zaliczony
I14	Webhook z niepoprawnym podpisem	Status 400, rezerwacja nadal `PENDING_PAYMENT`, brak e-maila	Zaliczony
I15	Webhook z treścią zmienioną po podpisaniu	Status 400, rezerwacja bez zmian	Zaliczony
I16	Webhook bez `reservationId` w metadanych	Status 400, komunikat `Missing reservationId metadata`	Zaliczony

5.2. Nominatim i proces geokodowania

Nominatim jest silnikiem geokodowania[4] opartym na OpenStreetMap. OpenStreetMap jest to mapa tworzona i utrzymywana przez społeczność, mająca na celu zachowanie danych i prawdziwości map. W aplikacji jest używany w celu zmiany danych adresu wpisanego przez użytkownika na współrzędne geograficzne. Następnie frontend, korzystając z map OpenStreetMap, wyświetla interaktywną mapę ze znacznikami dostępnych usług. Poza tym jest dostępny endpoint GET locations/cities, który wysyłając żądanie do serwisu Nominatim zwraca listę miast o nazwie wprowadzonej przez użytkownika. Po wybraniu miasta w filtrze usługi

[4] Geokodowanie jest to przypisanie współrzędnych geograficznych (długość, szerokość) danemu obiektowi z podanych danych adresowych. Dane te muszą znaleźć się w odpowiednich kolumnach (państwo, miasto, ulica, numer, kod pocztowy).

są wyświetlane ponownie, tym razem tylko te dostępne w tym mieście, tak samo jak i na interaktywnej mapie (którą trzeba zainicjalizować klikając przycisk „Pokaż mapę"). Do obsługi i wyświetlenia mapy jest używana biblioteka Leaflet, która daje możliwość wyświetlenia nowoczesnej mapy z funkcjami np. przybliżania. Nominatim to projekt OpenSource, więc jest darmowy, głównie dlatego zdecydowano się go użyć w aplikacji. Dla porównania, Google Geocoding API oferuje tylko 10 000 darmowych żądań, a kolejne są już płatne. Cennik można znaleźć pod tym adresem `https://developers.google.com/maps/billing-and-pricing/pricing#places-pricing`. Do używania publicznego API Nominatim potrzebny jest tzw. UserAgent, który w kodzie aplikacji trzeba podać, aby zidentyfikować aplikację, oraz kontakt email. Warto zaznaczyć, że limit użycia Nominatim to 1 żądanie na sekundę. W aplikacji użycie Nominatim występuje podczas tworzenia ogłoszenia. Użytkownik, który posiada dostęp do tworzenia ogłoszenia, po wpisaniu istniejącego adresu, i następnie po kliknięciu przycisku "Dodaj ogłoszenie", tworzy ogłoszenie. Wysyłane jest w tym momencie żądanie do serwera do kontrolera, który sprawdza czy użytkownik jest zalogowany i ma dostęp do tworzenia ogłoszeń, po czym kontroler odwołuje się do serwisu odpowiadającego za obsługę metod związanych z ogłoszeniami, a konkretnie do metody „addProduct()". W tej metodzie następuje komunikacja z NominatimService, który wysyła żądanie do API Nominatim, w celu zamienienia adresu ogłoszenia na współrzędne geograficzne. Współrzędne wracają do serwera w formacie JSON, a dalej są zapisywane do bazy danych w polach dołączonych do ogłoszenia. W przypadku wpisania nieprawidłowego adresu, albo adresu który nie został odnaleziony, geokodowanie nie zwróci żadnych współrzędnych geograficznych, co za tym idzie nie zostaną one zapisane w bazie. Na końcu całe ogłoszenie jest zapisywane do bazy danych, zważając wcześniej na poprawność walidacji danych. Diagram całego przebiegu znajduje się poniżej. Drugim przypadkiem użycia Nominatim w aplikacji jest wyszukiwarka miast. Użytkownik w panelu listy ogłoszeń, ma dostęp do wielu filtrów pozwalających na znalezienie odpowiedniego ogłoszenia. Jednym z nich jest filtr miast. Podczas wpisania nazwy miasta, gdy użytkownik przestanie ją wpisywać na 3 sekundy jest wyszukiwana lista maksymalnie 10 miast o podanej nazwie, wyświetlona za pomocą skomunikowania się z endpointem w backendzie odpowiedzialnym za komunikację z zewnętrznym serwisem Nominatim. Metoda serwisowa, buduje URL na podstawie otrzymanej nazwy miasta, który przykładowo wygląda tak: `https://nominatim.openstreetmap.org/search?q=Olsztyn&format=jsonv2&featureType=city&limit=10&accept-language=pl`. URL zawiera informacje takie jak, nazwa miasta, format danych, limit miast, które mogą się wyświetlić (w celu poprawienia sprawności aplikacji), oraz język w którym wyszukiwane jest miasto. API zwraca listę miast w formacie JSON z danymi takimi jak np. współrzędne geograficzne, nazwa oraz wyświetlana nazwa. Wyświetlana nazwa zawiera również województwo, kraj lub powiat. Użytkownik klikając w dane miasto wysyła kolejne żądanie do backendu, który zwraca listę ogłoszeń w tym mieście wraz z użyciem paginacji.



Rysunek 5.1: Diagram przepływu geokodowania adresu podczas tworzenia ogłoszenia

5.3. Detale techniczne aplikacji

5.3.1. JpaSpecificationExecutor

`JpaSpecificationExecutor` jest to wbudowany interfejs w Spring Data JPA, pozwalający na tzw. dynamiczne wywoływanie zapytań do bazy. Polega na budowaniu warunków wyszukiwania w czasie rzeczywistym. Pozwala składać warunki filtrowania, w zależności od kryteriów wybranych przez użytkownika, bez konieczności implementacji osobnej metody repozytorium dla każdej kombinacji filtrów. W klasie definiującej metody filtrowania używa się interfejsu `Specification`. Interfejs ten posiada jedną kluczową metodę wyglądającą tak:

```
Predicate toPredicate(Root<T> root,
                      CriteriaQuery<?> query,
                      CriteriaBuilder cb);
```

1. `Root<T>` – encja, z której pobierane są dane (odpowiednik `FROM`), po nim odwołujemy się do pól. Inaczej jest używany do określenia, w jakiej tabeli będą wyszukiwane dane.
2. `CriteriaQuery` – reprezentuje budowane całe zapytanie do bazy danych, zapisane nie w postaci zapytania np. SQL, ale w postaci obiektu. Budowanie obiektu zamiast czystego zapytania SQL daje kilka korzyści. Pierwszą z nich jest to, że błędy w budowie zapytania są wykrywane już na etapie kompilacji, a nie dopiero w trakcie działania aplikacji, jak w przypadku zapytania zapisanego jako ciąg znaków. Drugą korzyścią jest ochrona przed SQL injection. W przypadku gdy zapytanie do bazy danych było reprezentowane w postaci ciągu znaków, ktoś mógłby zmienić zapytanie w niestandardowy sposób, wpisując odpowiednie znaki.
3. `CriteriaBuilder` – obiekt zawierający metody, które odpowiadają operatorom i funkcjom SQL (`cb.equal` → `=`, `cb.like` → `LIKE`, `cb.and` → `AND`, `cb.or` → `OR`, `cb.lessThan` → `<`). Służy do budowania warunków zapytania.

Za przykład takiej metody może posłużyć poniższy kod:

```
Specification<Service> hasCategory =
    (root, query, cb) -> cb.equal(root.get("category"), "MASSAGE");
```

Gdzie `Service` to klasa encji, dla której wykonywane będzie zapytanie. Parametry: `root`, `query`, `cb` – odpowiadają wcześniej opisanym: `Root`, `CriteriaQuery` oraz `CriteriaBuilder`. Metoda wyszukuje wszystkie rekordy, w których wartość w kolumnie `category` jest równa słowu `MASSAGE`.

5.3.2. Zabezpieczenia

Każdy endpoint w aplikacji jest oznaczony odpowiednimi adnotacjami zabezpieczającymi go przed wykorzystaniem przez nieupoważnione osoby. Przykładami takich adnotacji są: @PreAuthorize("hasAuthority('USER')"), która dopuszcza tylko zalogowanych użytkowników o uprawnieniach `USER`, oraz `@PermitAll`, która oznacza dopuszczenie całego ruchu sieciowego do tego endpointu. Ponadto dodatkowym zabezpieczeniem jest klasa `SecurityConfig`, która filtruje zapytania otrzymywane od klienta lub z zewnątrz i na podstawie zdefiniowanych w niej wytycznych określa, czy zapytanie na dany endpoint jest dla niego dopuszczalne. Jeżeli zapytanie nie posiada wystarczających uprawnień, czyli np. nie zawiera prawidłowego tokenu JWT, to filtr je odrzuca. Takie działanie zapewnia metoda `securityFilterChain`, w której te zasady są zdefiniowane.

Kolejnym zabezpieczeniem w aplikacji jest tzw. CORS[26]. „CORS to skrót od Cross-Origin Resource Sharing, czyli mechanizmu umożliwiającego bezpieczny dostęp do zasobów między różnymi źródłami. Oznacza to, że strona internetowa może załadować zasoby (np. skrypty, obrazy czy style CSS) z innej domeny, ale tylko wtedy, gdy ta domena wyrazi na to zgodę". W klasie konfiguracyjnej Spring definiujemy adresy URL, które będą mogły się kontaktować z serwerem. W aplikacji jest to `http://localhost:5173`, ze względu na to, że aplikacja na tym etapie jest dostępna tylko lokalnie, a klient działający na Vite jest skonfigurowany na tym adresie. Ponadto klasa pozwala również na określenie, jakie typy żądań mogą wysyłać do serwera wskazane wcześniej adresy URL, jak również jakie nagłówki. W aplikacji pozwolenie otrzymały typy: `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `OPTIONS`, oraz nagłówki: `Authorization`, `Content-Type`, `Accept`, `Origin`, `X-XSRF-TOKEN`, `X-CSRF-TOKEN`. Nagłówek `Authorization` przenosi token JWT, `Origin` zawiera adres, z którego wysyłane jest żądanie (na jego podstawie przeglądarka i serwer weryfikują CORS), a dwa ostatnie przenoszą token CSRF, służący do ochrony przed atakami typu Cross-Site Request Forgery.

5.3.3. Tabela kluczowych endpointów

Tabela 5.3: Kluczowe endpointy aplikacji

Metoda	**Endpoint**	**Rola**	**Opis**
auth-controller			
POST	`/auth/signin`	GOŚĆ	Logowanie użytkownika, zwraca token JWT
POST	`/auth/register`	GOŚĆ	Rejestracja nowego użytkownika
product-controller			
GET	`/products`	GOŚĆ	Pobranie listy produktów (usług) z filtrowaniem
POST	`/products`	PROVIDER	Dodanie nowego produktu
GET	`/products/{productId}`	GOŚĆ	Pobranie szczegółów wybranego produktu
PATCH	`/products/{productId}`	PROVIDER	Częściowa edycja produktu
DELETE	`/products/{productId}`	PROVIDER	Usunięcie produktu
GET	`/products/my-products`	PROVIDER	Pobranie produktów należących do zalogowanego użytkownika
product-offer-controller			
POST	`/offers/{productId}`	PROVIDER	Dodanie oferty do wskazanego produktu
GET	`/offers/{offerId}`	GOŚĆ	Pobranie szczegółów oferty
PUT	`/offers/{offerId}`	PROVIDER	Edycja oferty
DELETE	`/offers/{offerId}`	PROVIDER	Usunięcie oferty
reservation-controller			
POST	`/reservations`	USER	Utworzenie rezerwacji przez zalogowanego użytkownika
POST	`/reservations/guest`	GOŚĆ	Utworzenie rezerwacji przez gościa (bez konta)
GET	`/reservations/{offerId}`	GOŚĆ	Pobranie terminów rezerwacji dla danej oferty
GET	`/reservations/my-reservations`	USER	Pobranie rezerwacji zalogowanego użytkownika
GET	`/reservations/by-product/{productId}`	PROVIDER	Pobranie rezerwacji dla wskazanego produktu
DELETE	`/reservations/cancel/{reservationId}`	USER	Anulowanie rezerwacji przez klienta
DELETE	`/reservations/revoke/{reservationId}`	PROVIDER	Odwołanie rezerwacji przez właściciela produktu

Tabela przedstawia najważniejsze endpointy aplikacji, z pominięciem tych związanych ze Stripe, fakturami, kategoriami oraz nominatim. Ze względu na to, że ich proces wcześniej został szczegółowo opisany, lub są mało istotne.

Rozdział 6

Instrukcja obsługi

6.1. Instrukcja uruchomienia aplikacji

6.1.1. Wymagania wstępne

Do uruchomienia aplikacji niezbędne są następujące narzędzia:
— **Docker** oraz **Docker Compose**
— **Git** do pobrania kodu źródłowego,
— darmowe konto w serwisie **Stripe** wraz z narzędziem **Stripe CLI** (do obsługi płatności),
— konto pocztowe umożliwiające wysyłanie wiadomości przez SMTP (np. Gmail), jeśli mają być wysyłane powiadomienia e-mail.
— **Node.js** (wraz z menedżerem pakietów **npm**) do uruchomienia aplikacji klienckiej,
Aplikacja działa w kontenerach, więc nie jest wymagana lokalna instalacja Javy, ani bazy danych. Aplikacja kliencka uruchamiana jest lokalnie i wymaga środowiska Node.js.

6.1.2. Pobranie kodu źródłowego

Należy pobrać kod serwera:

```
git clone https://github.com/Barcik04/diploma_thesis
cd diploma_thesis
git checkout final
```

Repozytorium na github jest prywatne, przed klonowaniem należy się skontaktować z twórcą aplikacji Igorem Barcikowskim, email: Igor.bb00@gmail.com, w celu nadania dostępu. Backend w github jest na branch „final", a kod kliencki na branch „frontend".

6.1.3. Konfiguracja zmiennych środowiskowych

Konfiguracja aplikacji (hasła, klucze API, sekrety) znajduje się w pliku `.env`. Plik ten nie jest dołączony do repozytorium, ponieważ zawiera dane uwierzytelniające, których nie należy publikować. W repozytorium znajduje się natomiast plik wzorcowy `.env.example`, który należy skopiować do nowostworzonego pliku .env i go uzupełnić.

Tabela 6.1: Zmienne środowiskowe aplikacji

Zmienna	Opis
`POSTGRES_DB`	Nazwa bazy danych tworzonej w kontenerze PostgreSQL
`POSTGRES_USER`	Nazwa użytkownika bazy danych
`POSTGRES_PASSWORD`	Hasło użytkownika bazy danych

Zmienna	Opis
SPRING_DATASOURCE_URL	Adres JDBC bazy danych używany przez aplikację (host db to nazwa usługi w Docker Compose)
SPRING_DATASOURCE_USERNAME	Nazwa użytkownika, którym aplikacja łączy się z bazą (musi odpowiadać POSTGRES_USER)
SPRING_DATASOURCE_PASSWORD	Hasło, którym aplikacja łączy się z bazą (musi odpowiadać POSTGRES_PASSWORD)
JWT_SECRET	Sekret do podpisywania tokenów JWT, długi losowy ciąg znaków
MAIL_USERNAME	Adres konta pocztowego, z którego wysyłane są wiadomości
MAIL_PASSWORD	Hasło aplikacji do konta pocztowego (nie zwykłe hasło do konta)
STRIPE_SECRET_KEY	Tajny klucz API Stripe w trybie testowym (sk_test_...)
STRIPE_WEBHOOK_SECRET	Sekret podpisu webhooka Stripe (whsec_...)

Sekret JWT

Sekret można wygenerować poleceniem:

```
openssl rand -base64 48
```

Uzyskaną wartość należy wpisać do zmiennej JWT_SECRET.

Konfiguracja Stripe

Płatności w aplikacji obsługuje serwis Stripe. Na potrzeby uruchomienia lokalnego wystarczy konto w trybie testowym, które nie wymaga podawania danych firmy ani realnych środków:

1. Należy założyć darmowe konto na stronie stripe.com i włączyć tryb testowy (*Test mode*).
2. W panelu Stripe, w sekcji kluczy API (*Developers* → *API keys*), należy skopiować klucz tajny (sk_test_...) i wpisać go do zmiennej STRIPE_SECRET_KEY.

Webhook Stripe w środowisku lokalnym

Stripe informuje aplikację o statusie płatności za pomocą webhooka, czyli żądania HTTP wysyłanego na endpoint /payments/webhook. Serwery Stripe nie mają dostępu do adresu localhost, dlatego w środowisku lokalnym wykorzystuje się narzędzie Stripe CLI, które odbiera zdarzenia ze Stripe i przekazuje je do aplikacji. Środowisko jest inicjalizowane przy starcie kontenera, używając przy tym STRIPE_SECRET_KEY, automatycznie nawiązując połączenie z środowiskiem testowym Stripe użytkownika.

Do poprawnej weryfikacji podpisu webhooka aplikacja potrzebuje sekretu whsec_.... Jest to sekret generowany przez narzędzie Stripe CLI, które w projekcie działa w osobnym kontenerze (stripe-cli). Sekret jest wypisywany w logach kontenera stripe-cli przy jego uruchomieniu, dlatego konfigurację należy wykonać w następujących krokach:

1. Uzupełnić w pliku .env zmienną STRIPE_SECRET_KEY, a w zmiennej STRIPE_WEBHOOK_SECRET wpisać wartość tymczasową.
2. Uruchomić aplikację poleceniem:

   ```
   docker compose up --build
   ```

3. Odczytać sekret webhooka z logów kontenera stripe-cli:

```
docker compose logs stripe-cli
```

W logach pojawi się komunikat zawierający wartość w postaci `whsec_...`.

4. Wpisać odczytany sekret do zmiennej `STRIPE_WEBHOOK_SECRET` w pliku `.env`.
5. Zrestartować kontener aplikacji, aby wczytał nową wartość:

```
docker compose up -d --force-recreate app
```

Sekret jest wypisywany w logach przy każdym uruchomieniu kontenera `stripe-cli`, dlatego w razie potrzeby można go ponownie odczytać tym samym poleceniem. Po wykonaniu powyższych kroków webhook działa poprawnie, a aplikacja otrzymuje informacje o opłaconych płatnościach.

6.1.4. Środowisko VS Code (aplikacja kliencka)

Aplikacja kliencka uruchamiana jest lokalnie, najwygodniej w edytorze **Visual Studio Code**. Przed rozpoczęciem należy uruchomić serwer zgodnie z poprzednimi punktami (`docker compose up -build`), ponieważ aplikacja kliencka komunikuje się z API pod adresem `http://localhost:8080`.

Pobranie kodu aplikacji klienckiej

Kod klienta znajduje się w tym samym repozytorium, na gałęzi `frontend`. Najlepiej sklonować go do osobnego katalogu:

```
git clone https://github.com/Barcik04/diploma_thesis
cd diploma_thesis
git checkout frontend
```

Instalacja zależności

W terminalu, w głównym katalogu projektu klienckiego, należy wykonać:

```
npm install
```

Polecenie pobiera biblioteki wymienione w pliku `package.json` (m.in. React, React Router, biblioteki Stripe i Leaflet) do katalogu `node_modules`. Wykonuje się je przy pierwszym uruchomieniu oraz po zmianie zależności.

Konfiguracja klucza publicznego Stripe

Aplikacja kliencka wymaga własnego pliku `.env`, umieszczonego w głównym katalogu projektu klienckiego (obok pliku `package.json`). Plik ten nie jest dołączony do repozytorium. Należy go utworzyć i wpisać do niego klucz publiczny Stripe:

1. Zalogować się do panelu Stripe i upewnić się, że włączony jest tryb testowy (*Test mode* / *Sandbox*).
2. W sekcji kluczy API (*Developers* → *API keys*) skopiować klucz publiczny (*Publishable key*, `pk_test_...`).
3. W katalogu projektu klienckiego utworzyć plik `.env` o następującej zawartości:

```
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_ ...
```

Vite udostępnia aplikacji wyłącznie zmienne z prefiksem `VITE_`. Klucz publiczny służy tylko do inicjalizacji formularza płatności, a klucz tajny pozostaje na serwerze. Klucz publiczny musi pochodzić z tego samego konta i trybu Stripe co klucz tajny (`STRIPE_SECRET_KEY`) użyty w serwerze, w przeciwnym razie płatność nie powiedzie się.

Uruchomienie aplikacji klienckiej

npm run dev

Po uruchomieniu aplikacja jest dostępna pod adresem `http://localhost:5173`.

Konfiguracja poczty

Aplikacja wysyła wiadomości e-mail przez protokół SMTP. W przypadku konta Gmail należy włączyć weryfikację dwuetapową, a następnie wygenerować *hasło aplikacji* i wpisać je do zmiennej `MAIL_PASSWORD`. Nie należy używać zwykłego hasła do konta. Jeżeli dane poczty nie zostaną skonfigurowane, aplikacja uruchomi się, ale wiadomości e-mail nie będą wysyłane.

6.1.5. Uruchomienie

Po uzupełnieniu pliku `.env` aplikację uruchamia się jednym poleceniem z katalogu głównego projektu:

```
docker compose up --build
```

Przy pierwszym uruchomieniu zostaną pobrane obrazy i zbudowane kontenery, co może zająć kilka minut. Po zakończeniu aplikacja jest dostępna pod adresami:
— aplikacja kliencka: `http://localhost:5173`,
— API serwera: `http://localhost:8080`,
— dokumentacja API (Swagger): `http://localhost:8080/swagger-ui.html`.

6.1.6. Testowanie płatności

W trybie testowym Stripe nie są pobierane prawdziwe środki. Do opłacenia rezerwacji można użyć testowej karty `4242 4242 4242 4242` z dowolną przyszłą datą ważności i dowolnym kodem CVC. Po poprawnym opłaceniu Stripe CLI wyświetli w terminalu przekazane zdarzenie, a aplikacja zaktualizuje status płatności.

6.1.7. Zatrzymanie aplikacji

```
docker compose down        # zatrzymanie kontenerow
docker compose down -v     # zatrzymanie i usuniecie danych bazy
```

6.1.8. Uwagi dotyczące bezpieczeństwa

Plik `.env` został dodany do `.gitignore` i nie powinien być publikowany ani przesyłany razem z kodem. Klucze zawarte w pliku `.env.example` są jedynie wartościami przykładowymi. Przed wdrożeniem aplikacji w środowisku produkcyjnym należy wygenerować nowe sekrety, zastąpić klucze testowe Stripe kluczami produkcyjnymi oraz zmienić adres dozwolony w konfiguracji CORS na docelową domenę.

Rozdział 7

Frontend

7.1. Technologia

React 19 + TypeScript, Vite, react-router-dom 7, Stripe (stripe-js, react-stripe-js), Leaflet do wyświetlania mapy. Aplikacja działa w architekturze SPA[1], która komunikuje się z backendem za pomocą REST/JSON. Aplikację zaimplementowano w React z użyciem TypeScript, który dzięki statycznemu typowaniu pozwala wykrywać błędy już na etapie kompilacji, oraz narzędzia Vite, które zapewnia szybkie budowanie się serwera.

7.2. Struktura projektu

Projekt jest rozdzielony na foldery, które oddzielają odpowiedzialności w aplikacji. W podfolderze „api" znajdują się pliki TypeScript, które posiadają logikę metod wywołujących endpointy z backendu. Przykładem tych metod jest np. metoda login, która komunikuje się z endpointem odpowiedzialnym za obsługę logowania i wydawania tokenu JWT użytkownikom. Sam token JWT w tej metodzie - po otrzymaniu go od serwera, zostaje zapisany w schowku lokalnym (localstorage) po stronie klienta. W taki sposób jest on zapisany w przeglądarce użytkownika niżeli w aplikacji. Przy wylogowaniu, token się usuwa ze schowka. Metody mają też obsługę błędów przy np. błędnym wpisaniu hasła przez użytkownika, komunikując go odpowiednią wiadomością, która w przypadku metody login brzmi następująco: „Nieprawidłowy email lub hasło". Korzyścią takiego rozdzielenia metod endpointów od kodu stron, poprawia skalowalność i łatwość późniejszego wprowadzania poprawek. W wielu przypadkach ta sama metoda jest wywoływana w wielu komponentach aplikacji przez co, gdyby nie wcześniej opisana rozdzielność zadań, programista musiałby szukać i zmieniać metodę w każdym miejscu jej występowania. Kolejnymi plikami rozdzielonymi od innych są tzw. komponenty funkcyjne,

czyli fragmenty interfejsu JSX, który jest wykorzystywany w wielu różnych miejscach aplikacji. Umieszono je w katalogu „components". Przykładem komponentu może być występujący w prawie każdej współczesnej aplikacji webowej navbar, czyli inaczej pasek nawigacji. W aplikacji jest on wyświetlany u góry strony, mając kilka przycisków przekierowujących użytkownika na profil użytkownika lub do strony głównej aplikacji. Poza tym występują tam przyciski które wyświetlają formularz rejestracji lub logowania użytkownika. Występuje również nieco bardziej skomplikowany komponent komunikujący się z Stripe API za pomocą biblioteki Stripe. Renderuje on formularz płatności kartą oraz obsługuje wysłanie płatności do Stripe. Następnie istnieje

katalog page, który przechowuje kod html generujący wygląd i interakcje na stronie, jak również

[1] Single-page application (SPA) to aplikacja webowa, która działa w ramach jednego dokumentu HTML i zamiast przeładowywać całą stronę przy każdej akcji użytkownika, dynamicznie aktualizuje tylko wybrane elementy interfejsu, treść i inne dane. Dzięki temu pracuje o wiele płynniej i szybciej

wywołuje metody zaimplementowane w katalogu api. W środku istnieją funkcje definiujące co ma się stać np. po kliknięciu przez użytkownika danego przycisku. Istnieją również dwa ostat-

nie katalogi, odpowiadające za wizualny wygląd aplikacji. Są to katalogi: „styles" i „photos". Pierwszy przechowuje kod css, odpowiadający za wygląd, kolor wielkość i animacje w aplikacji, ale również za dopasowanie wielkości strony do wielkości ekranu, na którym jest wyświetlana aplikacja. Robi to za pomocą adnotacji @media, w której opisuje się zasady w przypadku gdy strona będzie mniejsza niż określona ilość pikseli. Drugim folderem jest folder przechowujący zdjęcia i ikony. Nie są to zdjęcia wstawione przez użytkownika podczas tworzenia ogłoszenia, lecz zdjęcia które na stałe widnieją w aplikacji. Np. ikona domu reprezentującego powrót do strony głównej po jej kliknięciu. W katalogu głównym „src" istnieją jeszcze dwie ważne klasy

odpowiadające za nawigację i Routing. Definiują one, gdzie użytkownik zostanie przekierowany po użyciu danej drogi przypisanej do danego obiektu w kodzie np. napisu lub przycisku.

7.3. Ochrona tras

Za ochronę dostępnością tras odpowiada strona backendu, podczas wywoływania żądania, serwer zwraca odpowiedni komunikat wraz ze statusem (401/403), po czym klient wyświetla go w interfejsie. Oznacza to, że niezalogowany użytkownik może wejść w stronę profilu użytkownika, ale nie będzie mógł na nim wykonywać czynności ponieważ otrzyma komunikat o potrzebie zalogowania, aby odblokować te funkcjonalności. Takie podejście zapewni większą czytelność i zrozumienie aplikacji przez użytkowników oraz sprawi, że serwer decyduje o tym czy użytkownik ma dostęp do danych funkcjonalności co zwiększa bezpieczeństwo aplikacji.

7.4. Kalendarz i lista slotów

Do wyboru terminu rezerwacji zaimplementowano własny komponent kalendarza, bez użycia zewnętrznej biblioteki. Pozwoliło to dopasować go do modelu danych aplikacji, w którym oferta posiada cykliczną dostępność (dni tygodnia i przedziały godzin) oraz stały czas trwania usługi.

7.4.1. Siatka miesiąca

Siatka kalendarza budowana jest dla aktualnie wyświetlanego miesiąca. Liczbę dni miesiąca wyznacza się za pomocą obiektu `Date` (dzień zerowy następnego miesiąca to ostatni dzień bieżącego). Ponieważ w JavaScripcie niedziela ma indeks 0, a w polskim kalendarzu tydzień zaczyna się od poniedziałku, indeks pierwszego dnia miesiąca jest przesuwany, a przed pierwszym dniem wstawiane są puste komórki. Użytkownik przechodzi między miesiącami przyciskami "←" i "→".

7.4.2. Dostępność dni

Dzień jest aktywny (możliwy do kliknięcia), gdy oferta posiada wpis dostępności dla jego dnia tygodnia. Pozostałe dni są wyszarzone i przekreślone, co informuje użytkownika, że w tym dniu usługa nie jest świadczona.

7.4.3. Generowanie slotów

Po wybraniu dnia aplikacja generuje listę terminów na podstawie godzin rozpoczęcia i zakończenia dostępności oraz czasu trwania usługi (`duration`, w minutach). Godziny są zamieniane na minuty od północy, a kolejne sloty powstają z krokiem równym czasowi trwania usługi, dopóki slot mieści się w przedziale dostępności. Przykładowo, dla dostępności 09:00–12:00 i usługi trwającej 60 minut powstają sloty 09:00, 10:00 i 11:00. Metoda również zapewnia, że ostatnia usługa kończy się nie później niż koniec dostępności.

7.4.4. Zajęte terminy i wybór slotu

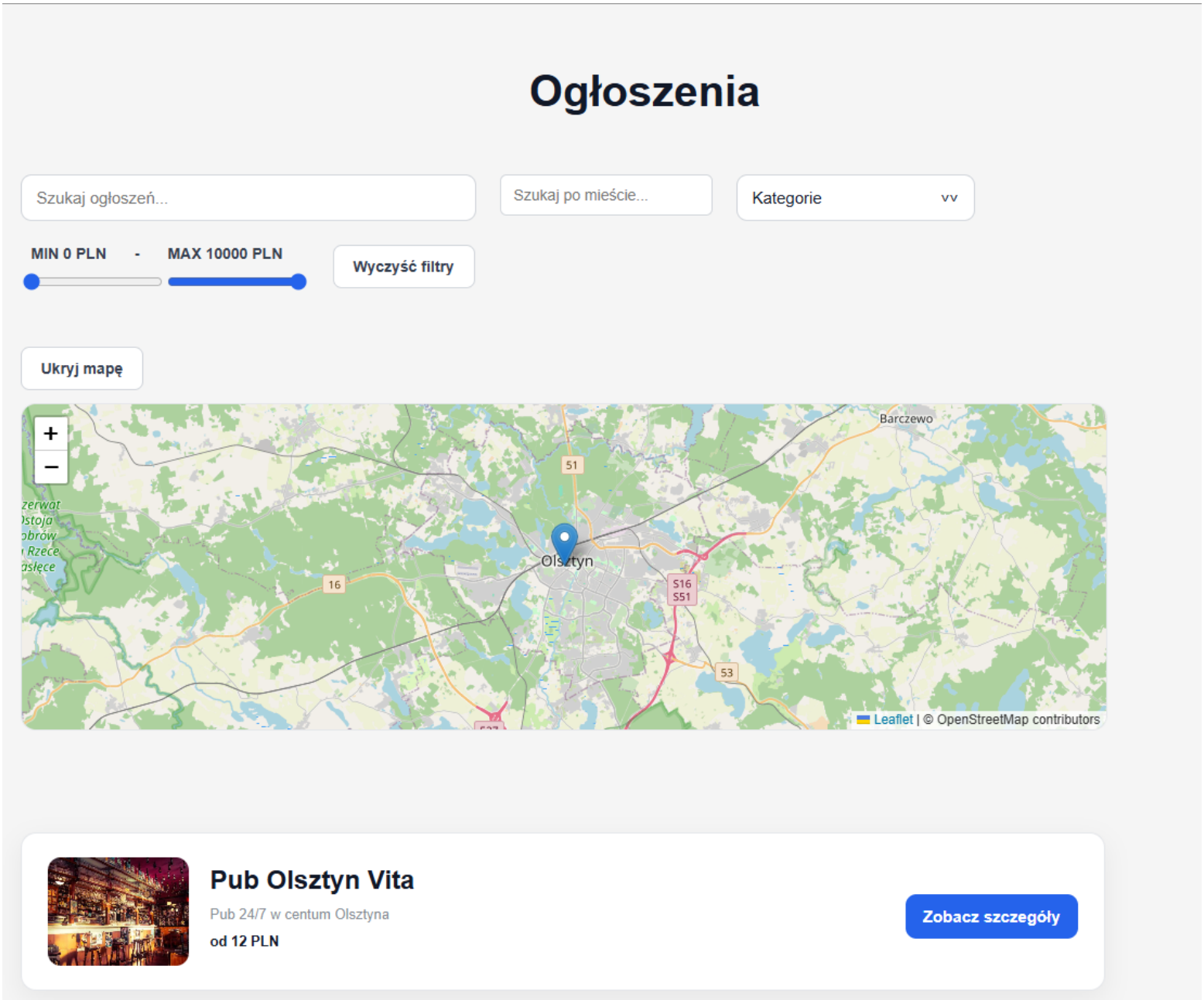
Rezerwacje danej oferty są pobierane z backendu i porównywane z każdym slotem po dacie i godzinie rozpoczęcia. Zajęte sloty są wyświetlane jako nieaktywne. Po wybraniu wolnego slotu użytkownik może podać dane do faktury, a następnie wybiera sposób płatności (gotówka lub karta). Niezalogowany gość jest przekierowywany do formularza rezerwacji gościa, a wybrany czas przekazywany jest w parametrze adresu URL.

7.4.5. Ograniczenia

Przeszłe dni bieżącego miesiąca nie są blokowane w interfejsie, a ich odrzucanie odbywa się po stronie backendu. Zajętość slotu sprawdzana jest wyłącznie po godzinie rozpoczęcia. Ostateczna walidacja dostępności terminu następuje w serwerze w momencie tworzenia rezerwacji.

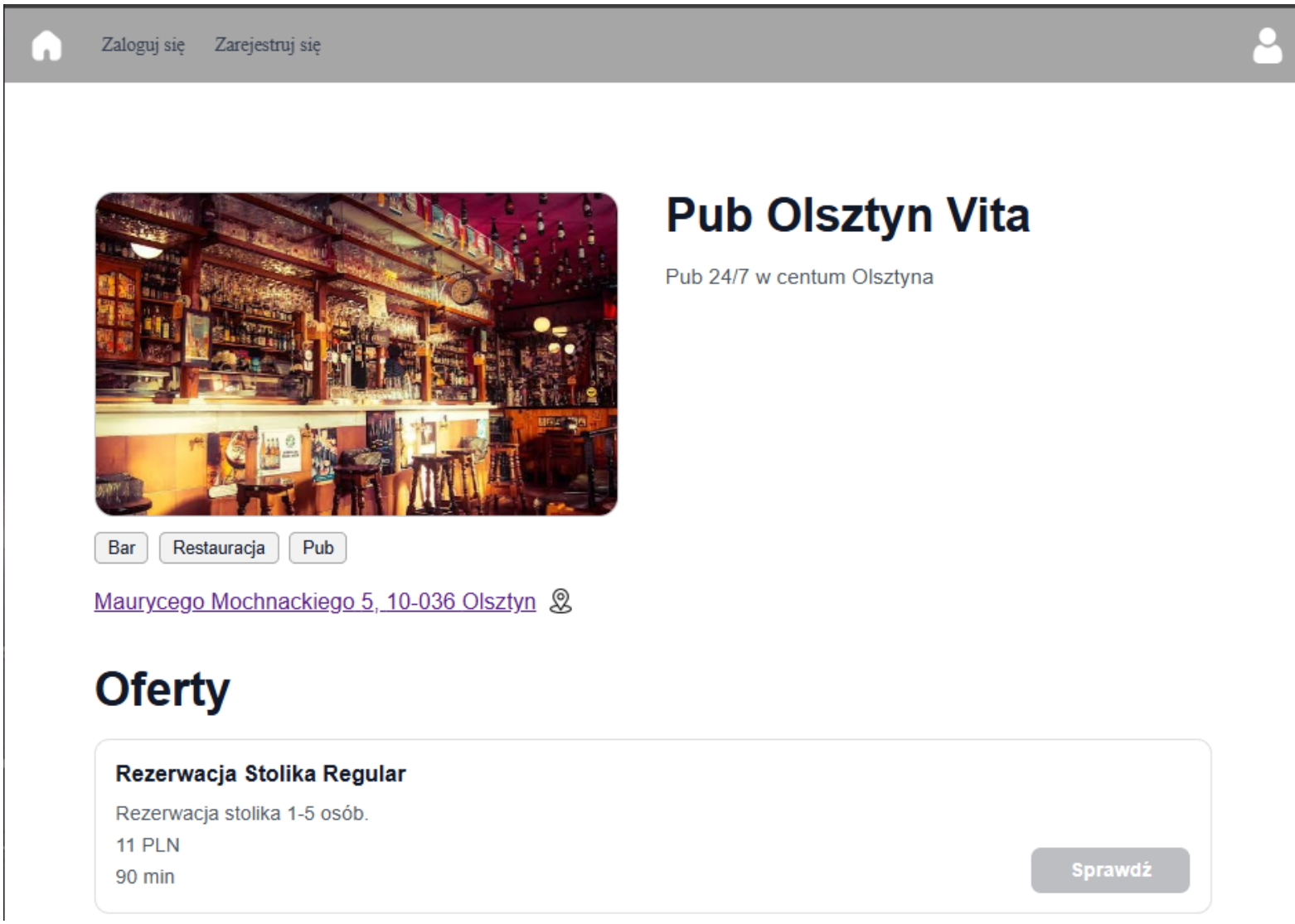
7.5. Pozostałe elementy interfejsu

Na stronie startowej ogłoszenia prezentowane są na interaktywnej mapie OpenStreetMap zbudowanej z użyciem bibliotek Leaflet i react-leaflet, gdzie każde ogłoszenie oznaczone jest pinezką. Wyniki można zawęzić filtrami (kategorie, zakres ceny), a wyszukiwanie miast działa z opóźnieniem (*debounce*), dzięki czemu zapytanie do serwera wysyłane jest dopiero po krótkiej przerwie w pisaniu, a nie przy każdym naciśnięciu klawisza. Lista ogłoszeń jest stronicowana po stronie backendu, co ogranicza ilość przesyłanych danych.



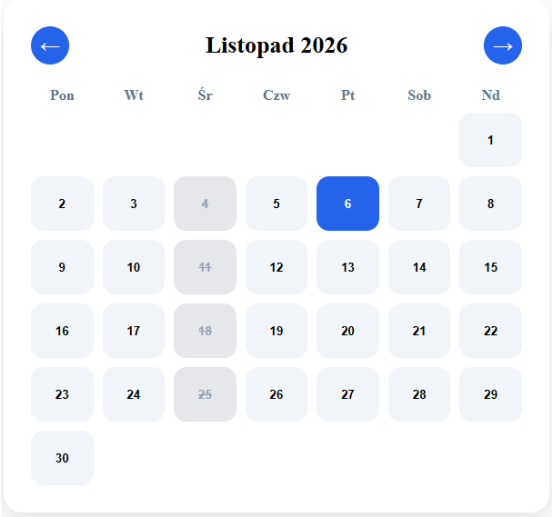
Rysunek 7.1: Zrzut ekranu z panelu wyszukiwarki ogłoszeń

Panel wyszukiwarki ogłoszeń (rys. 7.1) umożliwia użytkownikowi przeglądanie dostępnych ofert i filtrowanie ich według nazwy, miasta, kategorii oraz zakresu cenowego. Filtry można zresetować przyciskiem „Wyczyść filtry”. Lokalizacje ogłoszeń prezentowane są na interaktywnej mapie opartej na bibliotece Leaflet i danych OpenStreetMap, którą można pokazać lub ukryć przyciskiem. Poniżej mapy wyświetlana jest lista ogłoszeń, a każdy element zawiera zdjęcie (opcjonalnie), nazwę, krótki opis, cenę początkową oraz przycisk „Zobacz szczegóły”, który przenosi do widoku danego ogłoszenia.

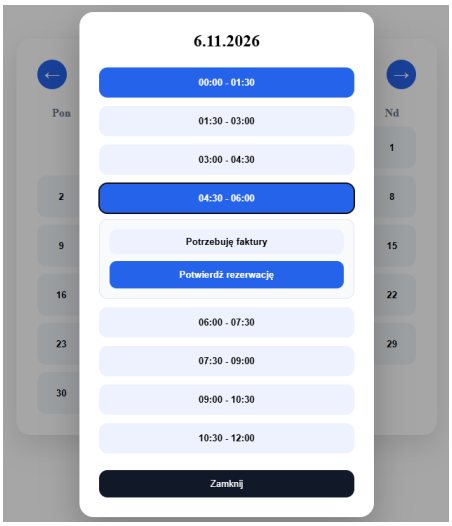


Rysunek 7.2: Zrzut ekranu z panelu danego ogłoszenia

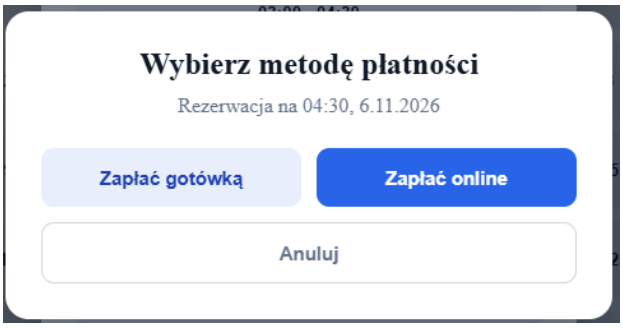
Rysunek 7.2 przedstawia widok pojedynczego ogłoszenia. U góry strony znajduje się pasek nawigacji, który jest dostępny na wszystkich podstronach aplikacji i umożliwia przejście do strony głównej, zalogowanie się, rejestrację oraz dostęp do konta użytkownika. Na pozostałych zrzutach ekranu pasek ten został pominięty w celu zmniejszenia rozmiaru wklejanych zdjęć. Widok ogłoszenia zawiera zdjęcie, nazwę, krótki opis, kategorie oraz adres lokalu. Poniżej wyświetlana jest sekcja „Oferty", w której każda oferta jest opisana nazwą, opisem, ceną i czasem trwania. Przycisk „Sprawdź" przenosi użytkownika do kalendarza dostępnych terminów danej oferty.



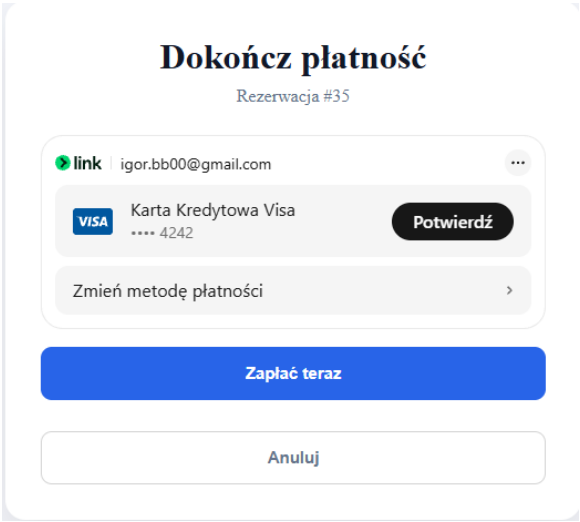
(a) Kalendarz dostępnych terminów



(b) Terminarz dostępnych godzin



(c) Okno wyboru płatności dla zalogowanych użytkowników



(d) Okno płatności

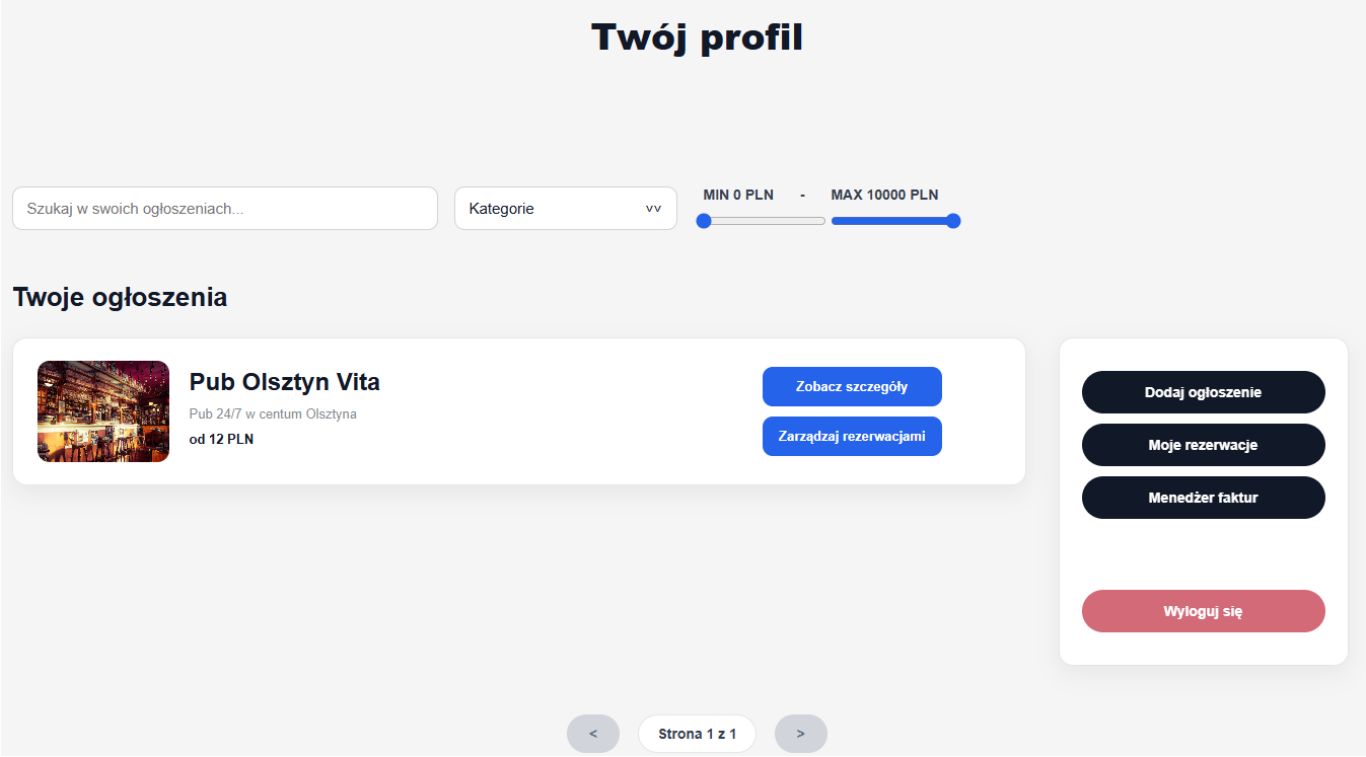
Rysunek 7.3: Zrzuty ekranu z procesu rezerwacji terminu i płatności

Rysunek 7.3 przedstawia widok wyboru terminu rezerwacji. Kalendarz dostępnych terminów (rys. 7.3a) wyświetla bieżący miesiąc z możliwością przełączania miesięcy za pomocą strzałek. Dni, w których usługa nie jest dostępna, są wyszarzone i nie można ich wybrać, a aktualnie wybrany dzień jest wyróżniony kolorem niebieskim. Po kliknięciu wybranego dnia otwiera się terminarz dostępnych godzin (rys. 7.3b), czyli okno z datą oraz listą przedziałów czasowych.

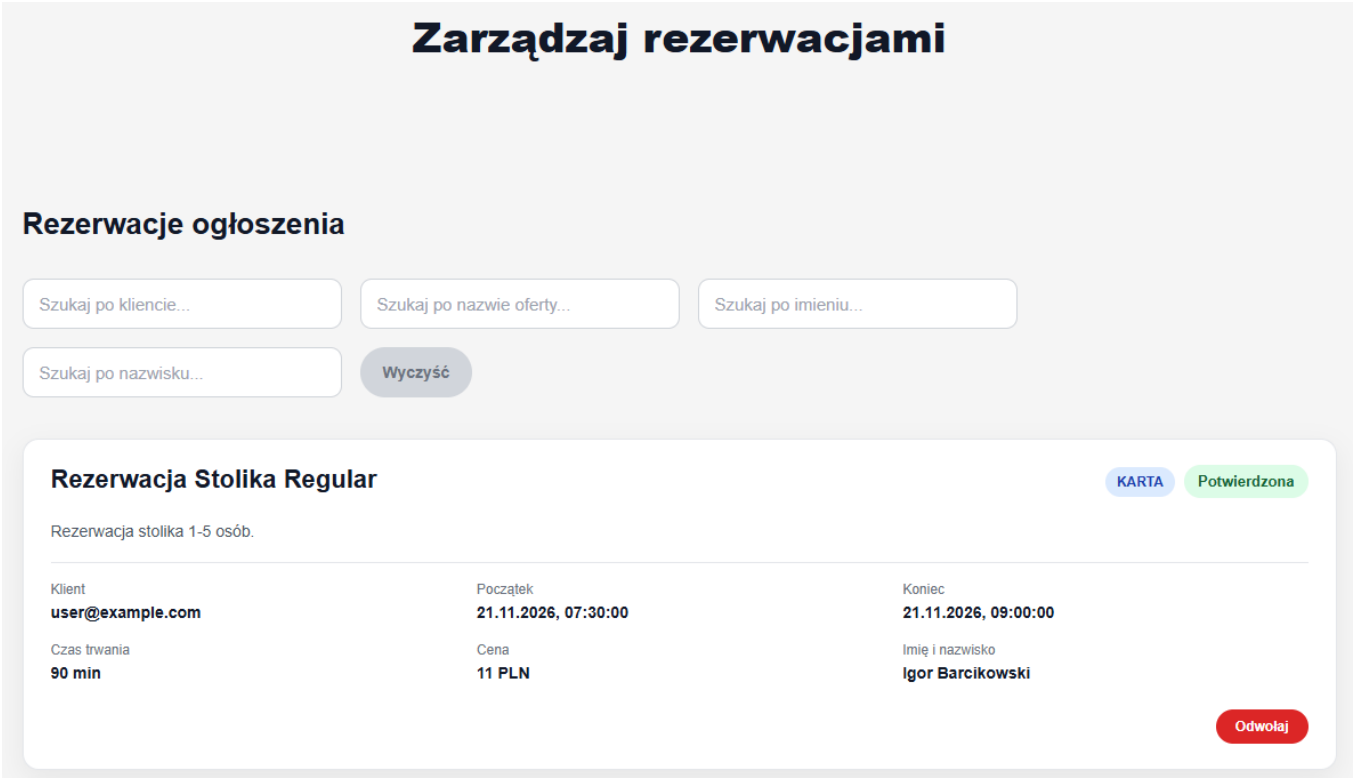
Po zarezerwowaniu danego terminu system blokuje go, dzięki czemu nie jest już widoczny jako dostępny dla innych użytkowników. Jeżeli zajęte zostaną wszystkie przedziały w danym dniu, dzień w kalendarzu zostaje wyszarzony.

Dalszy przebieg rezerwacji zależy od statusu użytkownika. Po kliknięciu „Potwierdź rezerwację" gość zostaje przekierowany do płatności. Użytkownikowi zalogowanemu wyświetla się natomiast okno wyboru metody płatności (rys. 7.3c). Zawiera ono informację o terminie rezerwacji oraz przyciski „Zapłać gotówką" (dostępny tylko wtedy, gdy właściciel lokalu dopuszcza taką formę płatności) i „Zapłać online", a także przycisk „Anuluj", który przerywa proces rezerwacji. Wybranie opcji „Potrzebuję faktury" powoduje wyświetlenie okien z formularzami, w których użytkownik uzupełnia dane niezbędne do jej wystawienia.

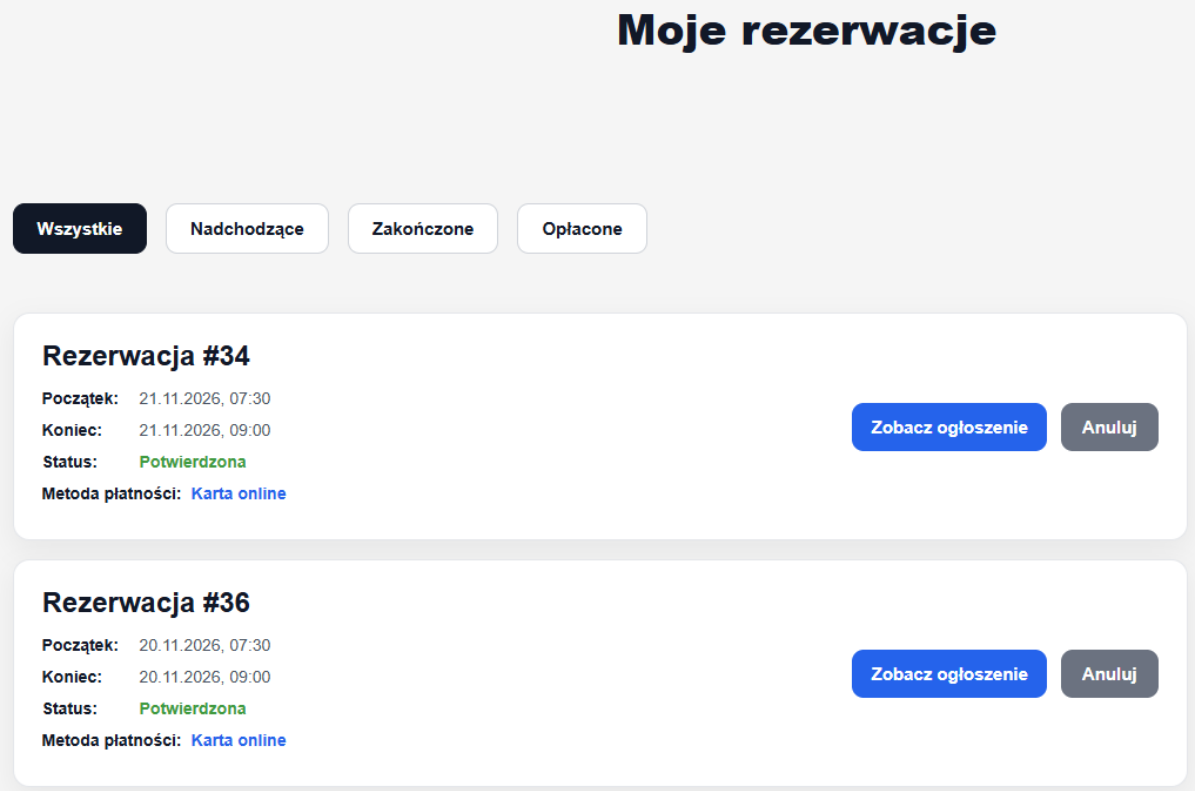
Wybór płatności online, a w przypadku gościa kliknięcie „Potwierdź rezerwację", otwiera okno płatności (rys. 7.3d). Wyświetla ono formularz płatności obsługiwany przez system Stripe, w którym użytkownik może skorzystać z zapisanej metody płatności, np. karty kredytowej, albo wybrać inną opcję przyciskiem „Zmień metodę płatności". Transakcję zatwierdza przycisk „Zapłać teraz", a przycisk „Anuluj" pozwala z niej zrezygnować. Po pomyślnym opłaceniu rezerwacja zostaje potwierdzona. Jeżeli usługa jest bezpłatna, płatność jest pomijana, a rezerwacja zostaje potwierdzona od razu.



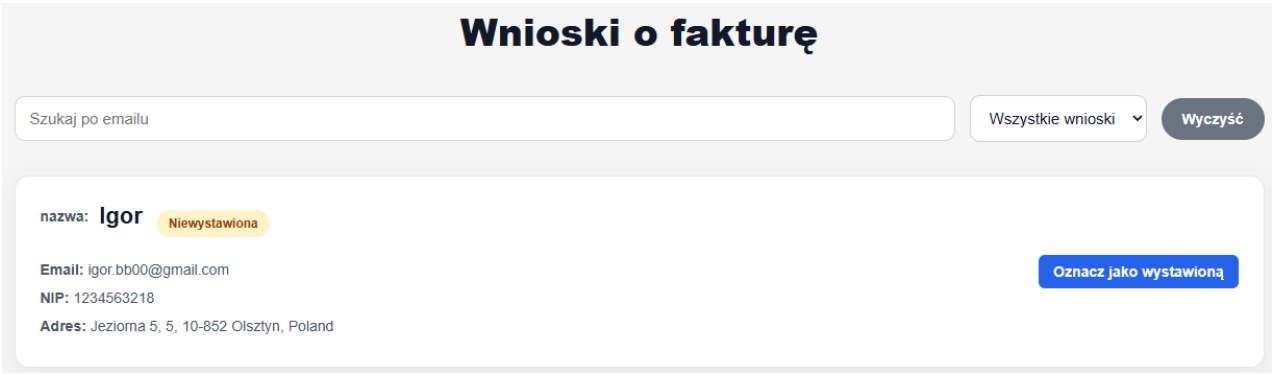
Rysunek 7.4: Zrzut ekranu z panelu profilu użytkownika



Rysunek 7.5: Zrzut ekranu z panelu zarządzania rezerwacjami



Rysunek 7.6: Zrzut ekranu z panelu rezerwacji użytkownika



Rysunek 7.7: Zrzut ekranu z panelu zarządzania fakturami

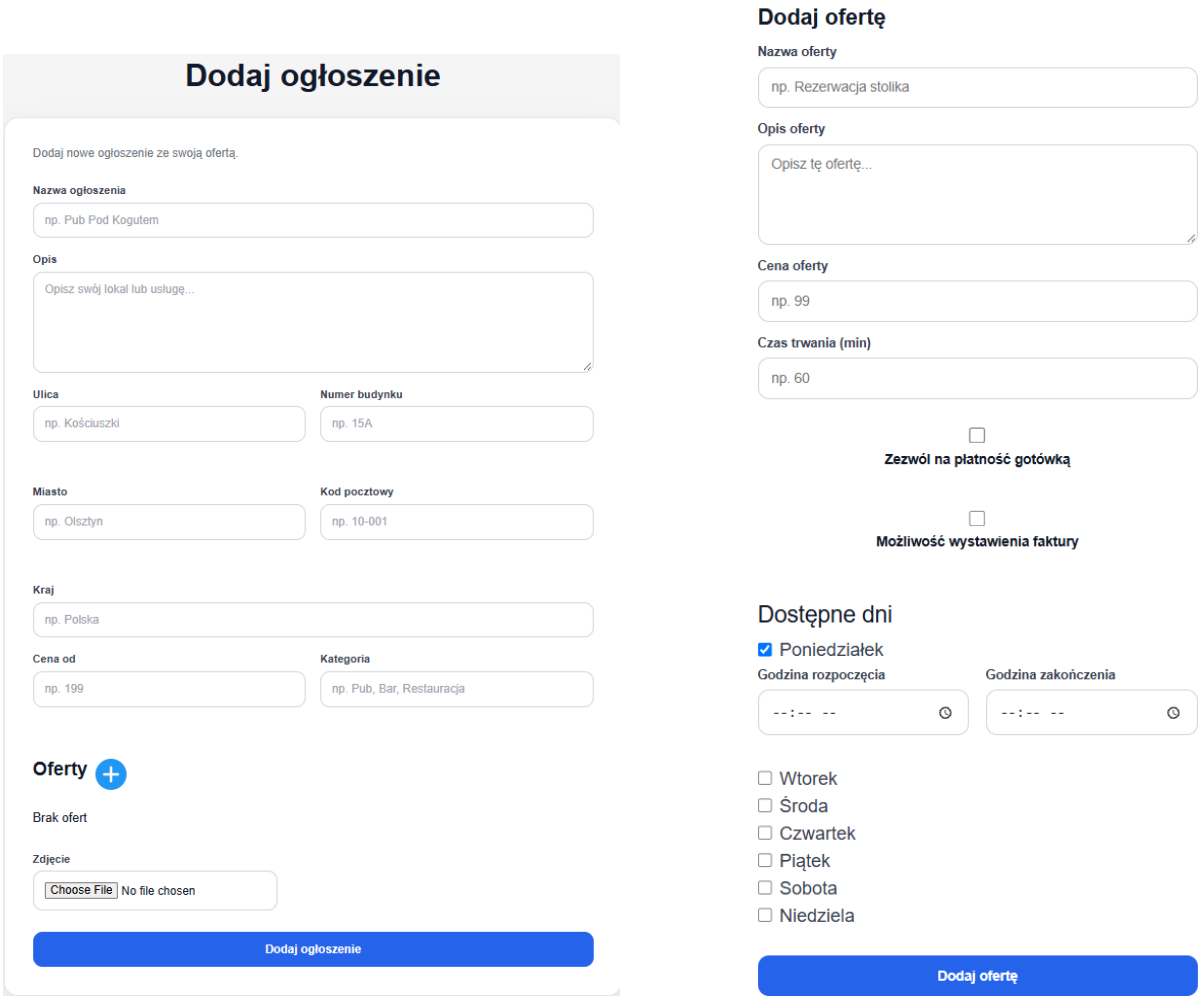
Funkcje aplikacji są zróżnicowane w zależności od statusu użytkownika. Gość może przeglądać i wyszukiwać ogłoszenia, wyświetlać ich szczegóły oraz zarezerwować termin z płatnością online. Zalogowany użytkownik ma dodatkowo dostęp do własnego profilu i historii rezerwacji. Funkcje związane z prowadzeniem ogłoszeń, czyli dodawanie ogłoszeń, zarządzanie rezerwacjami klientów i obsługa faktur, są dostępne wyłącznie dla użytkowników, którzy mają nadaną rolę usługodawcy przez administratora aplikacji.

Rysunek 7.4 przedstawia panel profilu użytkownika, który jest punktem wyjścia do pozostałych funkcji dla zalogowanych. W części głównej wyświetlana jest lista ogłoszeń stworzonych przez użytkownika z wyszukiwarką po nazwie, filtrem kategorii i zakresem cenowym oraz paginacją. Przy każdym ogłoszeniu znajdują się przyciski „Zobacz szczegóły" oraz „Zarządzaj rezerwacjami". Panel boczny zawiera przyciski „Dodaj ogłoszenie", „Moje rezerwacje", „Menedżer faktur" oraz „Wyloguj się". Sekcja „Twoje ogłoszenia" oraz przyciski „Dodaj ogłoszenie", „Zarządzaj rezerwacjami" i „Menedżer faktur" są przeznaczone dla usługodawców, natomiast „Moje rezerwacje" i „Wyloguj się" są dostępne dla każdego zalogowanego użytkownika.

Widok „Moje rezerwacje" (rys. 7.6) jest dostępny dla każdego zalogowanego użytkownika i służy do przeglądania własnych rezerwacji. Można je filtrować według statusu: wszystkie, nadchodzące, zakończone i opłacone. Każda rezerwacja jest pokazywana z terminem rozpoczęcia i zakończenia, statusem oraz metodą płatności. Przycisk „Zobacz ogłoszenie" prowadzi do ogłoszenia, którego dotyczy rezerwacja, a przycisk „Anuluj" pozwala ją odwołać.

Widok „Zarządzaj rezerwacjami" (rys. 7.5) jest przeznaczony wyłącznie dla usługodawcy i pokazuje rezerwacje złożone na jego ogłoszenie. Można je wyszukiwać według klienta, nazwy oferty, imienia i nazwiska. Każda rezerwacja zawiera nazwę i opis oferty, metodę płatności (np. karta), status (np. potwierdzona), dane klienta, termin, czas trwania oraz cenę. Usługodawca może odwołać rezerwację przyciskiem „Odwołaj".

Widok „Wnioski o fakturę" (rys. 7.7), dostępny z menedżera faktur, również jest przeznaczony dla usługodawcy. Wyświetla wnioski klientów, którzy wybrali opcję „Potrzebuję faktury", wraz z danymi niezbędnymi do jej wystawienia: nazwą, adresem e-mail, numerem NIP i adresem. Wnioski można wyszukiwać po adresie e-mail i filtrować według statusu, a po wystawieniu faktury usługodawca oznacza wniosek przyciskiem „Oznacz jako wystawioną".



(a) Formularz tworzenia ogłoszenia

(b) Formularz dodawania oferty

Rysunek 7.8: Zrzuty ekranu z formularzy tworzenia ogłoszenia i dodawania oferty

Formularz „Dodaj ogłoszenie” (rys. 7.8a) służy do dodania nowego ogłoszenia przez właściciela lokalu lub usługodawcę. Użytkownik wpisuje w nim nazwę i opis ogłoszenia, adres (ulica, numer budynku, miasto, kod pocztowy, kraj), cenę początkową oraz kategorię. W sekcji „Oferty” przyciskiem „+” dodaje poszczególne oferty, a następnie może dołączyć zdjęcie. Całość zatwierdza przycisk „Dodaj ogłoszenie”.

Formularz „Dodaj ofertę” (rys. 7.8b), otwierany po kliknięciu „+” w sekcji „Oferty”, służy do zdefiniowania pojedynczej oferty w ogłoszeniu. Użytkownik podaje nazwę, opis, cenę i czas trwania usługi w minutach oraz określa, czy dopuszcza płatność gotówką i czy oferuje możliwość wystawienia faktury. W sekcji „Dostępne dni” zaznacza dni tygodnia, w których oferta jest dostępna, i dla każdego z nich ustala godzinę rozpoczęcia i zakończenia. Całość zatwierdza przycisk „Dodaj ofertę”.

Rozdział 8

Zakończenie

8.1. Podsumowanie

Celem pracy było stworzenie uniwersalnej aplikacji webowej, która pozwala usługodawcom z różnych branż publikować ogłoszenia z ofertami i terminami dostępności, a klientom szybko wyszukiwać usługi i je rezerwować. Cel ten został zrealizowany.

Zrealizowano wszystkie założone wymagania funkcjonalne. Użytkownicy mogą się rejestrować i logować, a dostęp do zasobów jest kontrolowany za pomocą tokenów JWT i ról (GOŚĆ, USER, PROVIDER). Usługodawca może dodawać ogłoszenia z wieloma ofertami, ustalać dni i godziny dostępności oraz zarządzać rezerwacjami i wnioskami o faktury. Klient może przeglądać i filtrować ogłoszenia (także na mapie), rezerwować terminy oraz płacić online lub, o ile oferta na to pozwala, gotówką. Rezerwacje mogą składać również goście bez konta, a wznowienie przerwanej płatności umożliwia link wysyłany e-mailem. Płatności online obsługuje Stripe (w trybie testowym), a ich potwierdzanie odbywa się za pomocą webhooków. Aplikacja wysyła też wiadomości e-mail, m.in. potwierdzenia płatności i przypomnienia o wizycie.

Szczególną uwagę poświęcono poprawności i bezpieczeństwu rezerwacji. Podwójne rezerwacje tego samego terminu są blokowane na poziomie serwisu oraz dzięki izolacji transakcji `SERIALIZABLE`. Nieopłacone rezerwacje kartą wygasają automatycznie po 15 minutach, a dane kart nie trafiają na serwer aplikacji, lecz bezpośrednio do Stripe. Kluczowe procesy (rezerwacje, płatności, uwierzytelnianie i fakturowanie) zostały pokryte testami jednostkowymi i integracyjnymi, a testy integracyjne uruchamiane są na prawdziwej bazie PostgreSQL w kontenerze Docker. Całość można uruchomić jednym poleceniem dzięki Docker Compose.

8.2. Wnioski

Realizacja projektu pokazała, że oddzielenie warstwy klienta od serwera oraz podział backendu na kontrolery, serwisy i repozytoria ułatwia zarówno rozwój, jak i testowanie aplikacji. Wybór relacyjnej bazy danych okazał się uzasadniony ze względu na silnie powiązane ze sobą dane (ogłoszenia, oferty, rezerwacje, faktury), a mechanizm `JpaSpecificationExecutor` pozwolił zbudować elastyczne filtrowanie bez konieczności tworzenia ogromnej ilości metod repozytorium. Przekazanie obsługi płatności zewnętrznemu operatorowi uprościło aplikację i ograniczyło ryzyko związane z przetwarzaniem danych kart, jednak wymagało dokładnego zaprojektowania obsługi webhooków oraz stanów rezerwacji.

Zastosowane rozwiązania sprawdzają się w środowisku testowym i przy skali typowej dla projektu inżynierskiego. Wdrożenie produkcyjne wymagałoby jednak uzupełnienia systemu o elementy opisane poniżej, a także m.in. zastąpienia kluczy testowych Stripe produkcyjnymi oraz zmiany konfiguracji CORS na docelową domenę. Wdrożenie wymagałoby również własnego serwera lub usług chmurowych, co wiązałoby się z dodatkowymi kosztami.

8.3. Ograniczenia i możliwości rozwoju aplikacji

Zrealizowany system ma kilka ograniczeń, które wskazują kierunki dalszego rozwoju. Rola usługodawcy jest nadawana ręcznie w bazie danych, ponieważ aplikacja nie posiada panelu administratora. Uwierzytelnianie oparte na tokenach JWT nie umożliwia ich unieważniania po stronie serwera ani odświeżania (brak refresh tokena). System nie oferuje również resetowania hasła ani weryfikacji adresu e-mail. Zwroty i anulowanie płatności nie są obsługiwane automatycznie i wymagają ręcznej obsługi w systemie Stripe.

Naturalnymi kierunkami rozwoju są więc: panel administratora z procesem nadawania roli usługodawcy, mechanizm odświeżania i unieważniania tokenów, reset hasła i weryfikacja e-mail, automatyczne zwroty płatności po odwołaniu rezerwacji, integracja z systemami fakturowania (w tym KSeF) oraz aplikacja mobilna i obsługa wielu języków.

Bibliografia

[1] Wprowadzenie do Systemów Baz Danych, Elmasri Ramez, Navathe Shamkant B.; Helion; 2019.

[2] Język UML 2.0 w modelowaniu systemów informatycznych, Wrycza S., Marcinkowski B., Wyrzykowski K.; Helion; 2005.

[3] Inżynieria oprogramowania, Ian Sommerville; PWN; 2020.

[4] Java Podstawy. Wydanie XII, Cay Horstmann; Helion; 2022.

[5] React dokumentacja, `https://react.dev/learn`, dostęp: 17.03.2026.

[6] Spring JPA dokumentacja, `https://docs.spring.io/spring-data/jpa/reference/api/java/`, dostęp: 17.03.2026.

[7] PostgreSQL dokumentacja, `https://www.postgresql.org/docs/`, dostęp: 17.03.2026.

[8] Spring Security dokumentacja, `https://docs.spring.io/spring-security/reference/index.html`, dostęp: 17.03.2026.

[9] Spring Boot dokumentacja, `https://docs.spring.io/spring-boot/index.html`, dostęp: 17.03.2026.

[10] Nominatim dokumentacja, `https://nominatim.org/release-docs/develop/`, dostęp: 06.09.2026.

[11] TypeScript dokumentacja, `https://www.typescriptlang.org/docs/`, dostęp: 06.09.2026.

[12] react-leaflet dokumentacja, `https://react-leaflet.js.org/docs/start-introduction/`, dostęp: 06.09.2026.

[13] Stripe dokumentacja, `https://docs.stripe.com/`, dostęp: 06.09.2026.

[14] Stripe API dokumentacja, `https://docs.stripe.com/api?lang=java`, dostęp: 06.09.2026.

[15] Booking.com for Business, *Jak mogę otrzymać fakturę?*, `https://help.business.booking.com/hc/pl/articles/14862570197268-Jak-mog%C4%99-otrzyma%C4%87-faktur%C4%99`, dostęp: 20.09.2026.

[16] Tpay, *Jak Polacy kupują online 2024/2025 — ulubione metody płatności*, `https://tpay.com/user/assets/files_for_download/news/ulubione-metody-platnosci-online-raport_informacja%20prasowa.pdf`, dostęp: 20.09.2026.

[17] Fakturownia, *Krajowy System e-Faktur — czym jest i jak wygląda integracja z Fakturownią*, `https://fakturownia.pl/komunikaty/krajowy-system-e-faktur-czym-jest-i-jak-wyglada-integracja-z-fakturownia`, dostęp: 20.09.2026.

[18] DefinicjaFramework, *Definicja frameworku wikipedia*, `https://pl.wikipedia.org/wiki/Framework`, dostęp: 20.09.2026.

[19] DefinicjaRESTAPI, *Definicja REST API*, `https://www.ovhcloud.com/pl/learn/what-is-rest-api/`, dostęp: 20.09.2026.

[20] DefinicjaHTTP, *Definicja HTTP*, `https://ks.pl/slownik/co-to-jest-http`, dostęp: 20.09.2026.

[21] listaFrontendFramework, *Lista frameworków frontendowych*, `https://hackajob.com/talent/blog/top-10-frameworks-to-learn-in-2025-and-when-to-use-them`, dostęp: 20.09.2026.

[22] listaBackendFramework, *Lista frameworków backendowych*, `https://rockstardeveloperuniversity.com/best-backend-frameworks/`, dostęp: 20.09.2026.

[23] webhookDefinicja, *webhook Definicja*, `https://www.getresponse.pl/pomoc/webhook.html`, dostęp: 26.09.2026.

[24] MockitoFrameworkSite, *Mockito framework site*, https://site.mockito.org/, dostęp: 24.10.2026.
[25] Leafletbiblioteka, *Biblioteka Leaflet*, https://leafletjs.com/reference.html, dostęp: 31.10.2026.
[26] CORSdefinicja, *CORS definicja*, https://dhosting.pl/pomoc/baza-wiedzy/czym-jest-cors-i-jak-dziala/, dostęp: 03.11.2026.

Streszczenie

Praca przedstawia implementację aplikacji webowej do zarządzania rezerwacjami usług. Aplikacja pozwala usługodawcom z różnych branż publikować ogłoszenia z ofertami, cenami i terminami dostępności, a klientom wyszukiwać usługi, filtrować je (także na mapie) oraz rezerwować terminy, również bez zakładania konta. Część serwerową zrealizowano w Javie z użyciem Spring Boot jako REST API z uwierzytelnianiem opartym na tokenach JWT i rolami użytkowników, dane przechowywane są w bazie PostgreSQL, a interfejs użytkownika wykonano w React i TypeScript. Płatności online obsługuje Stripe w trybie testowym, a ich potwierdzanie odbywa się za pomocą webhooków. Do geokodowania adresów wykorzystano Nominatim, a do wysyłania powiadomień serwer SMTP. W pracy opisano założenia teoretyczne, projekt bazy danych, architekturę, kluczowe procesy (rezerwacja, płatność, logowanie), zabezpieczenia oraz wyniki testów jednostkowych i integracyjnych. Pokazano m.in. sposób ochrony przed podwójnymi rezerwacjami z użyciem izolacji transakcji `SERIALIZABLE`. Na koniec wskazano ograniczenia systemu i kierunki jego dalszego rozwoju.

Abstract

Application for Managing Reservations

The thesis presents the design and implementation of a universal web application for managing service reservations. The application allows service providers from different industries to publish listings with offers, prices and availability, while customers can search and filter services (including on a map) and book time slots, also without creating an account. The backend was implemented in Java using Spring Boot as a REST API with JWT-based authentication and user roles, data is stored in a PostgreSQL database, and the user interface was built with React and TypeScript. Online payments are handled by Stripe in test mode and confirmed by webhooks. Nominatim is used for address geocoding and an SMTP server for sending notifications. The thesis describes the theoretical background, the database design, the architecture, key processes (reservation, payment, login), security mechanisms, and the results of unit and integration tests. It shows, among other things, how double bookings are prevented using the `SERIALIZABLE` transaction isolation level. Finally, the limitations of the system and directions for further development are discussed.